3D visualization and processing with Napari

Thierry Pécot
Research Engineer
Biosit SFR UMS CNRS 3480 – Inserm 018
CZI Imaging Scientist

search Ctrl K

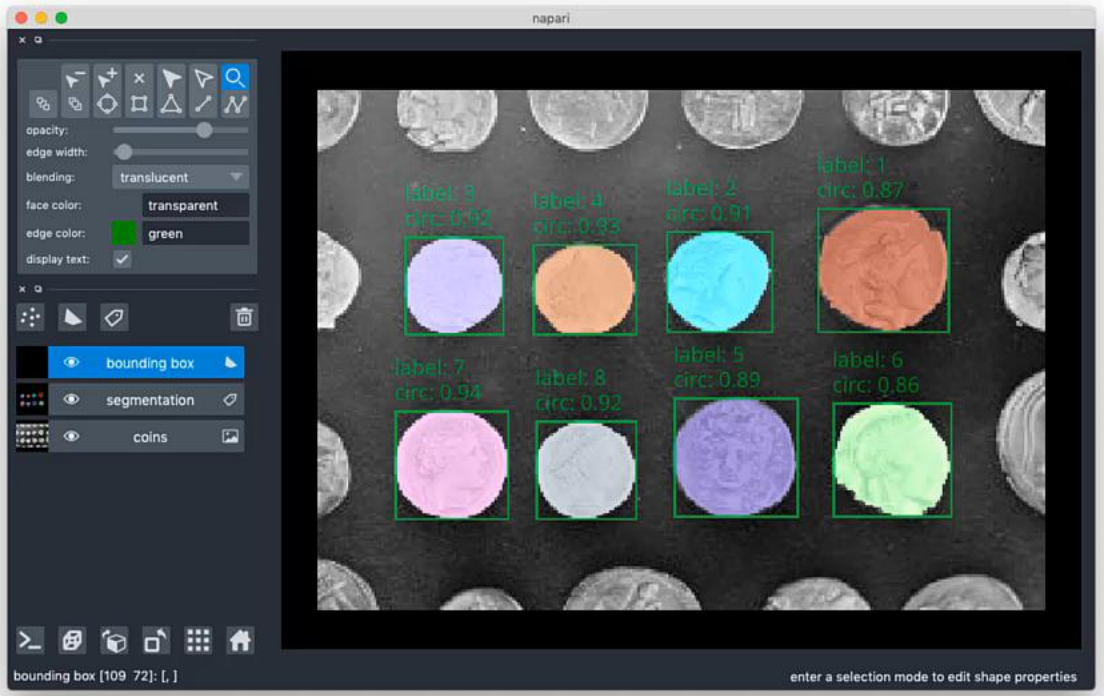
napari: a fast, interactive viewer for multi-dimensional images in Python

search Ctrl K

- Getting started
- napari tutorials
  - Annotation
  - Processing
  - Segmentation
    - Annotating segmentation with text and bounding boxes
  - Tracking
- How-to guides
- In-depth explanations
- Glossary
- napari workshops
- Sample databases
- Gallery

Annotating segmentation with text and bounding boxes

In this tutorial, we will use napari to view and annotate a segmentation with bounding boxes and text labels. Here we perform a segmentation by setting an intensity threshold with Otsu's method, but this same approach could also be used to visualize the results of other image processing algorithms such as object detection with neural networks.

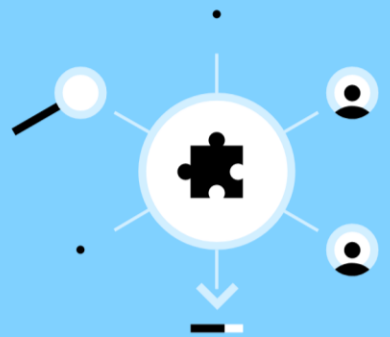


ON THIS PAGE

napari **hub**

Plugins

Discover, install, and share napari plugins



- Discover plugins that solve your image analysis challenges
- Learn how to install into napari

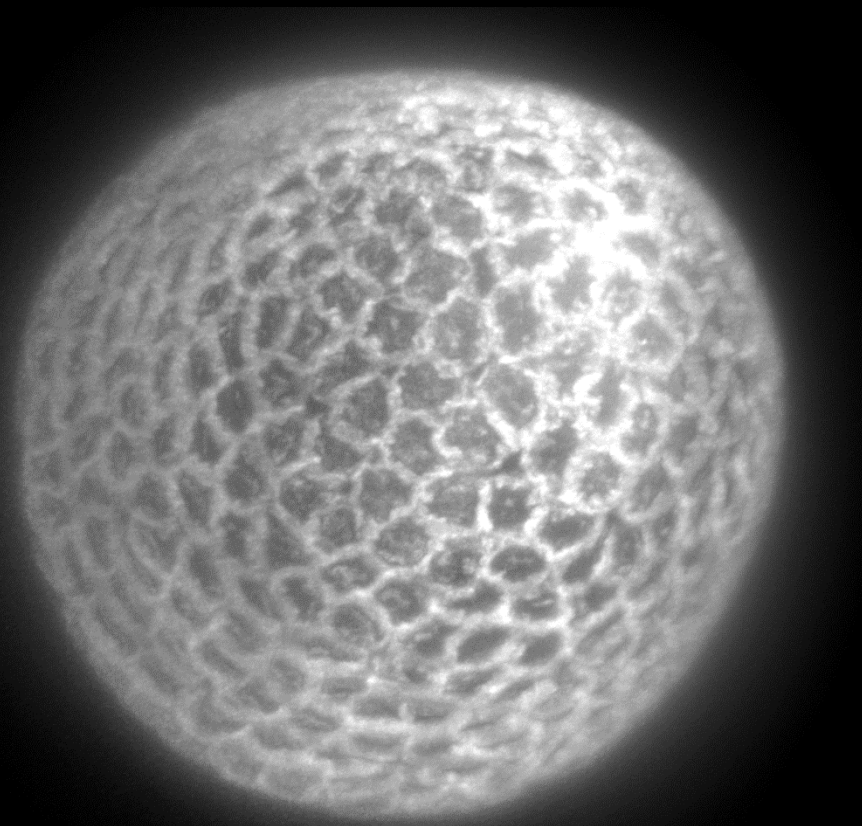
Share your image analysis tools with napari's growing community

Discover Plugins Browse all

Search for a plugin by keyword or author

In an **Anaconda prompt**:

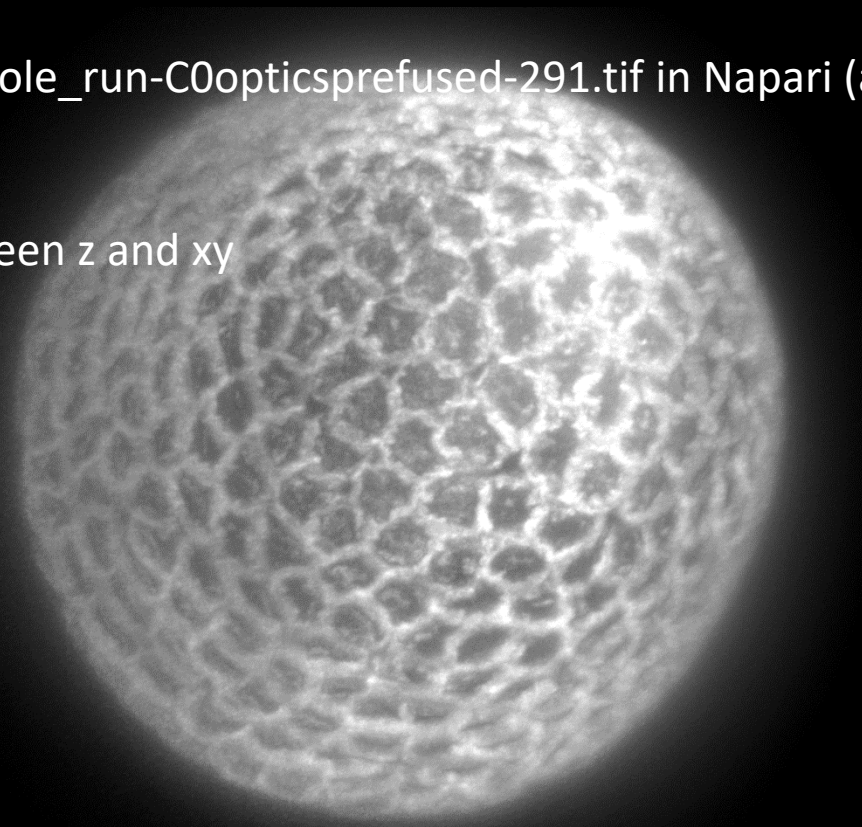
- mamba create -n NapariEnv python=3.10 openjdk=8 // create a virtual environment called NapariEnv
- mamba activate NapariEnv // activate the virtual environment NapariEnv
- pip install napari[all] // install Napari
- napari // open Napari



In an **Anaconda prompt**:

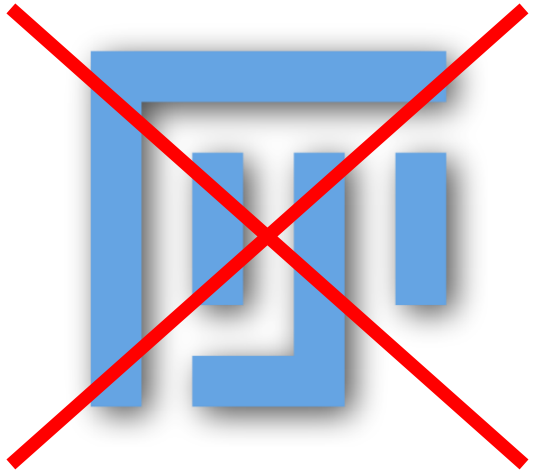
- mamba create -n NapariEnv python=3.10 openjdk=8 // create a virtual environment called NapariEnv
- mamba activate NapariEnv // activate the virtual environment NapariEnv
- pip install napari[all] // install Napari
- napari // open Napari

- Drag and drop Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif in Napari (available at https://zenodo.org/records/3981193)
- Visualize image in 2D and 3D, change scaling between z and xy
- Change contrast, change colormap, blending, ...
- Add Points layer
- Add shape layer
- Add a label layer, annotate 2 cells



Visualization

Visualization



- The **whole image** is loaded into the computer's **RAM** memory when opened with Fiji
- Stacks acquired with **lightsheet** microscopes = tens of GB

Visualization

- Open as virtual stack
- Save and visualize it with BigDataViewer

Visualization

- Open as virtual stack
- Save and visualize it with BigDataViewer

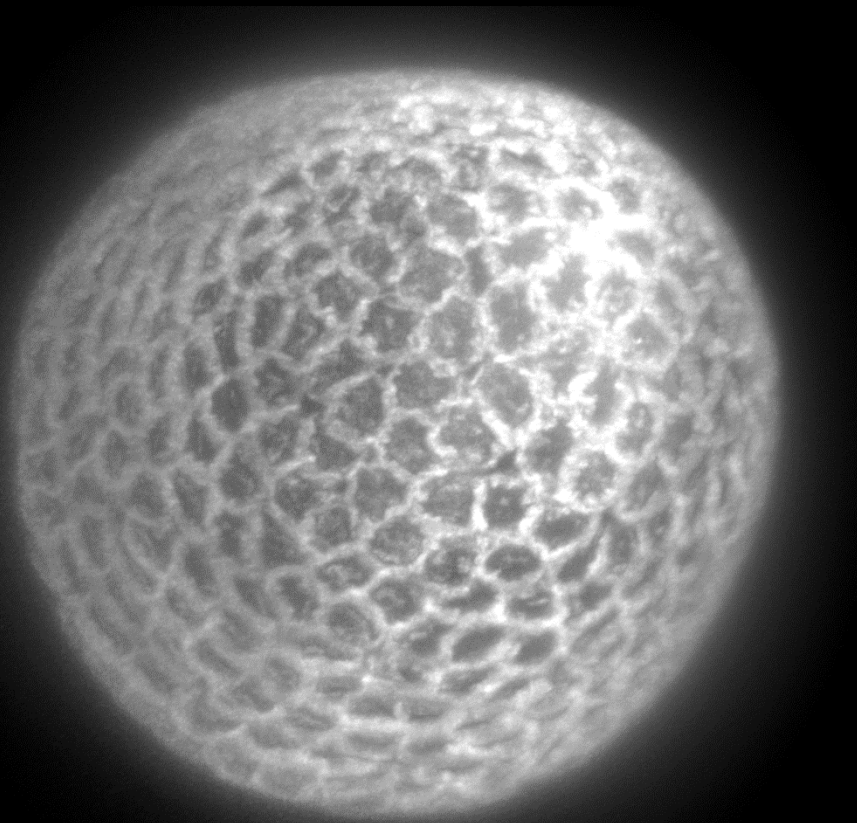
Zarr

napari

- Zarr format **splits data into chunks** so only **one or several chunks** are loaded **at once** into the computer's **RAM** memory
- **Napari**, a Python visualizer, allows to **visualize zarr data**

In a Miniforge **prompt**:

- mamba install -c ome bioformats2raw // install Python package bioformats2raw
- bioformats2raw Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.zarr // convert to zarr
- pip install napari-ome-zarr // install Napari plugin to open zarr images



In a Miniforge **prompt**:

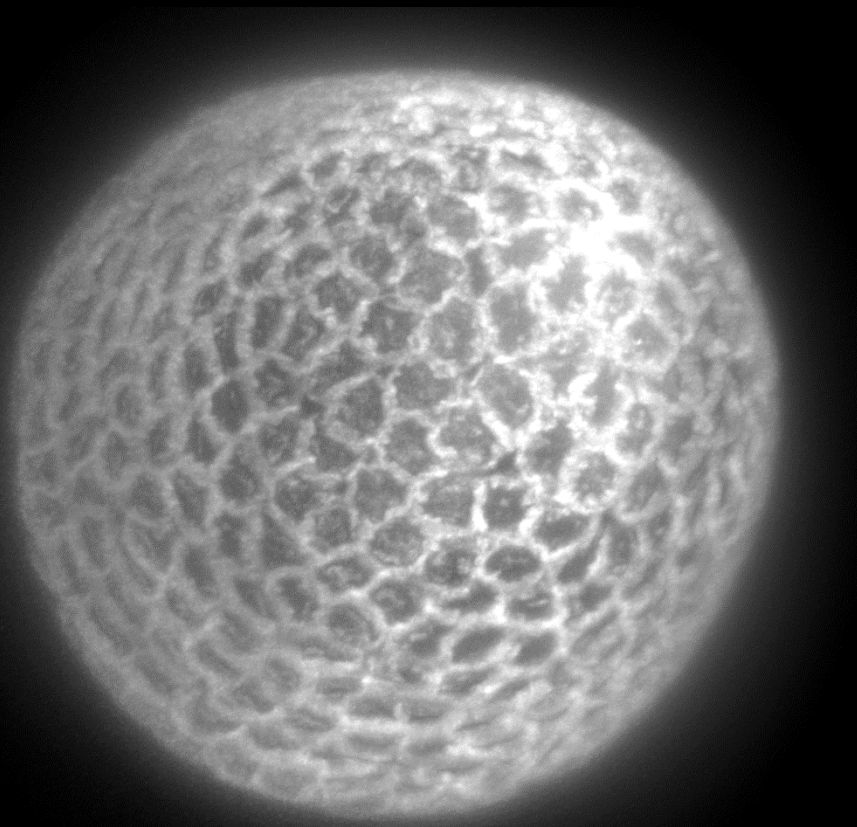
- mamba install -c ome bioformats2raw // install Python package bioformats2raw
- bioformats2raw Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.zarr // convert to zarr
- pip install napari-ome-zarr // install Napari plugin to open zarr images

- Compare the RAM usage between the tif and the zarr version of Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291 when opened in Napari

- https://napari.org/stable/tutorials/processing/dask.html

In a Miniforge **prompt**:

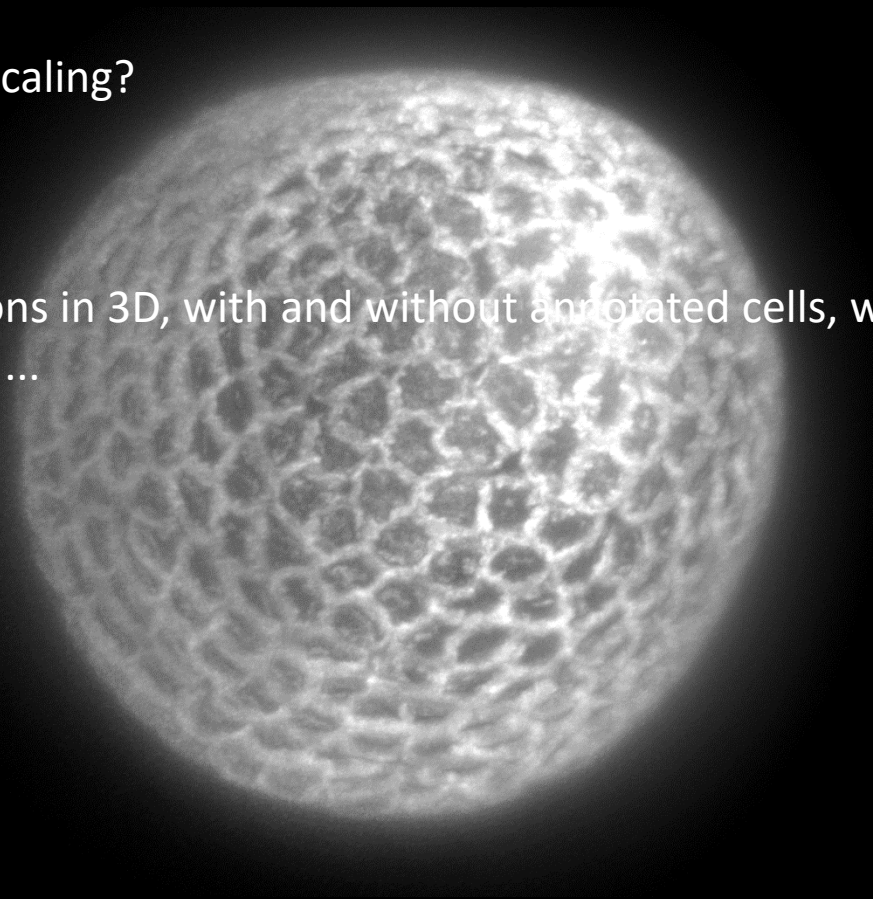
- pip install napari-bioformats napari-label-interpolator napari-animation // install Napari plugin to open proprietary formats, to interpolate labels, to do measurements from regions and to create videos



In a Miniforge **prompt**:

- pip install napari-bioformats napari-label-interpolator napari-animation // install Napari plugin to open proprietary formats, to interpolate labels, to do measurements from regions and to create videos

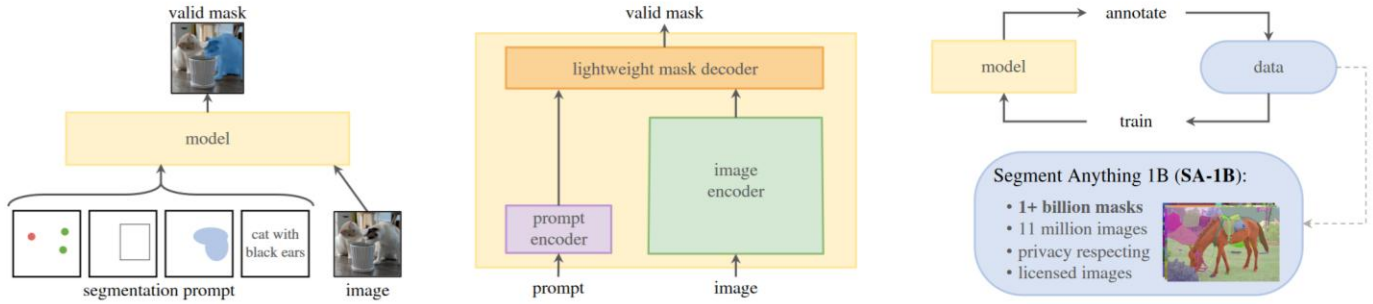
- Visualize image reconstruction in 3D, what about scaling?
- Annotate cells with the label interpolator plugin
- Create an animation with sections in 2D and sections in 3D, with and without annotated cells, with and without image, zooming in, zooming out, rotating, ...



FOUNDATION MODEL FOR INSTANCE SEGMENTATION

Segment Anything

Alexander Kirillov[1,2,4] Eric Mintun[2] Nikhila Ravi[1,2] Hanzi Mao[2] Chloe Rolland[3] Laura Gustafson[3]
Tete Xiao[3] Spencer Whitehead Alexander C. Berg Wan-Yen Lo Piotr Dollár[4] Ross Girshick[4]
[1]project lead [2]joint first author [3]equal contribution [4]directional lead
Meta AI Research, FAIR



(a) **Task**: promptable segmentation (b) **Model**: Segment Anything Model (**SAM**) (c) **Data**: data engine (top) & dataset (bottom)

Figure 1: We aim to build a foundation model for segmentation by introducing three interconnected components: a promptable segmentation *task*, a segmentation *model* (SAM) that powers data annotation and enables zero-shot transfer to a range of tasks via prompt engineering, and a *data* engine for collecting SA-1B, our dataset of over 1 billion masks.

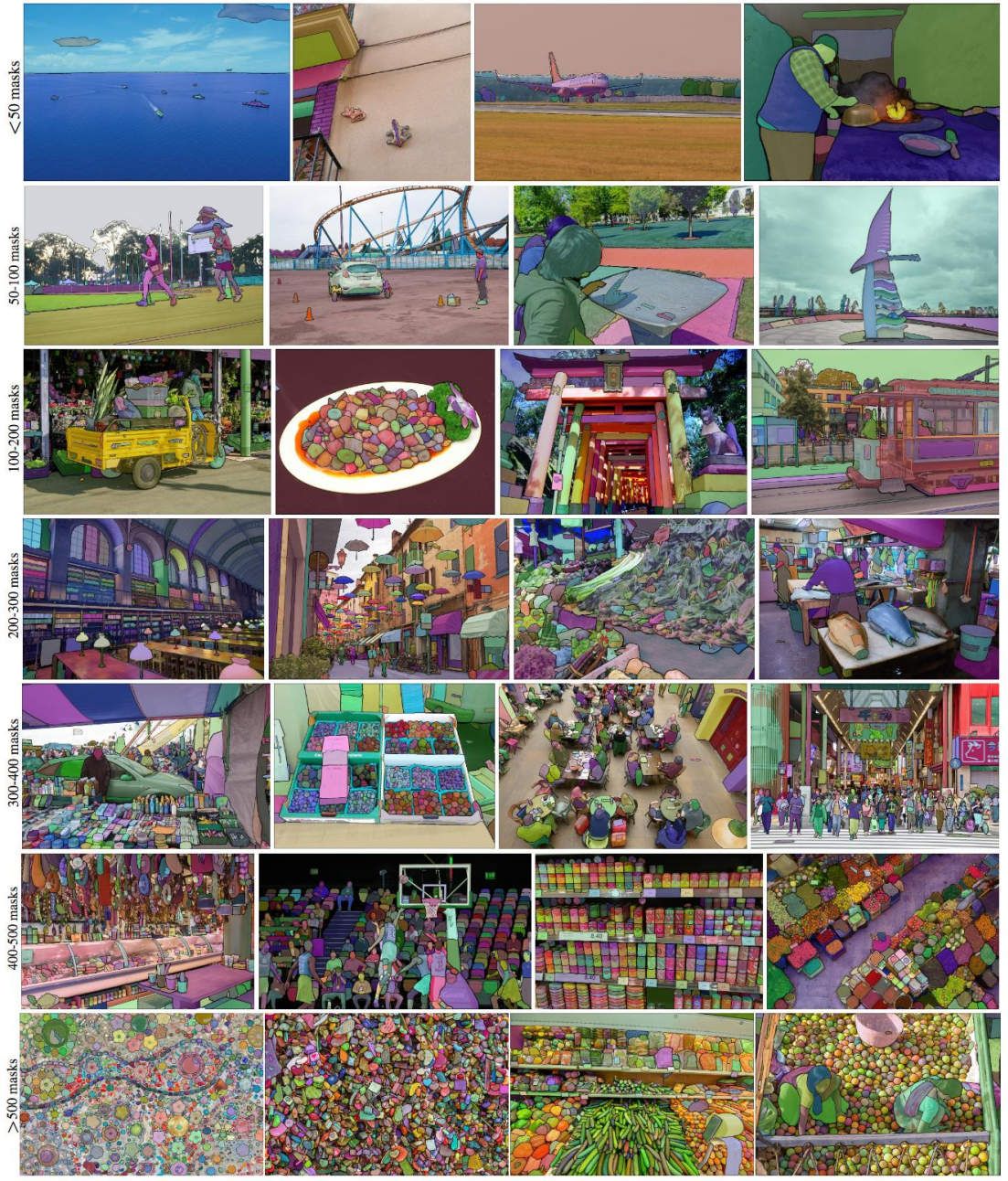


Figure 2: Example images with overlaid masks from our newly introduced dataset, **SA-1B**. SA-1B contains 11M diverse, high-resolution, licensed, and privacy protecting images and 1.1B high-quality segmentation masks. These masks were annotated *fully automatically* by SAM, and as we verify by human ratings and numerous experiments, are of high quality and diversity. We group images by number of masks per image for visualization (there are ~100 masks per image on average).

FOUNDATION MODEL FOR INSTANCE SEGMENTATION

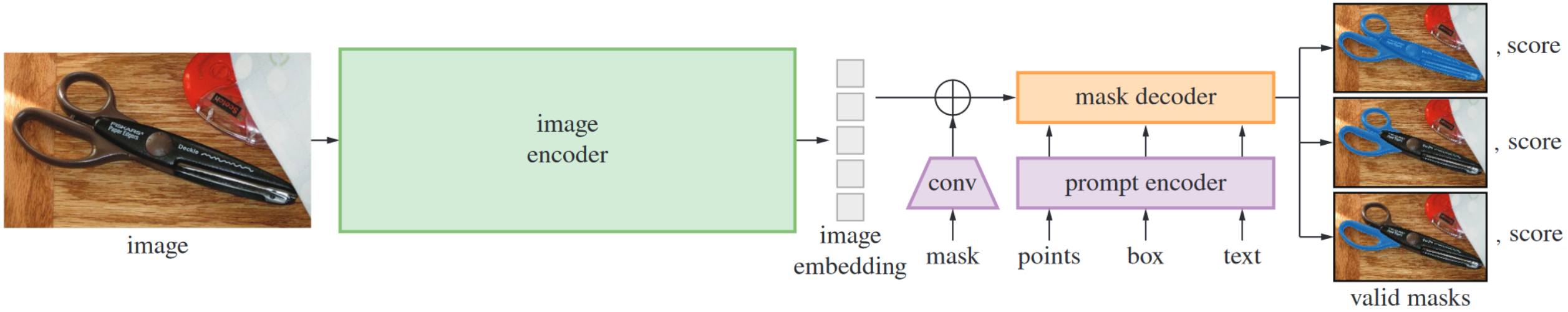


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

nature methods

Article

https://doi.org/10.1038/s41592-024-02580-4

Segment Anything for Microscopy

Received: 21 August 2023

Accepted: 26 November 2024

Published online: 12 February 2025

Check for updates

Anwai Archit[1], Luca Freckmann[1], Sushmita Nair[1], Nabeel Khalid[2,3], Paul Hilt[13], Vikas Rajashekar[2], Marei Freitag[1], Carolin Teuber[1], Melanie Spitzner[4], Constanza Tapia Contreras[4], Genevieve Buckley[5], Sebastian von Haaren[6], Sagnik Gupta[1], Marian Grade[4,7], Matthias Wirth[4,7,8,9,10], Günter Schneider[4,7,11], Andreas Dengel[2,3], Sheraz Ahmed[2] & Constantin Pape[1,12]

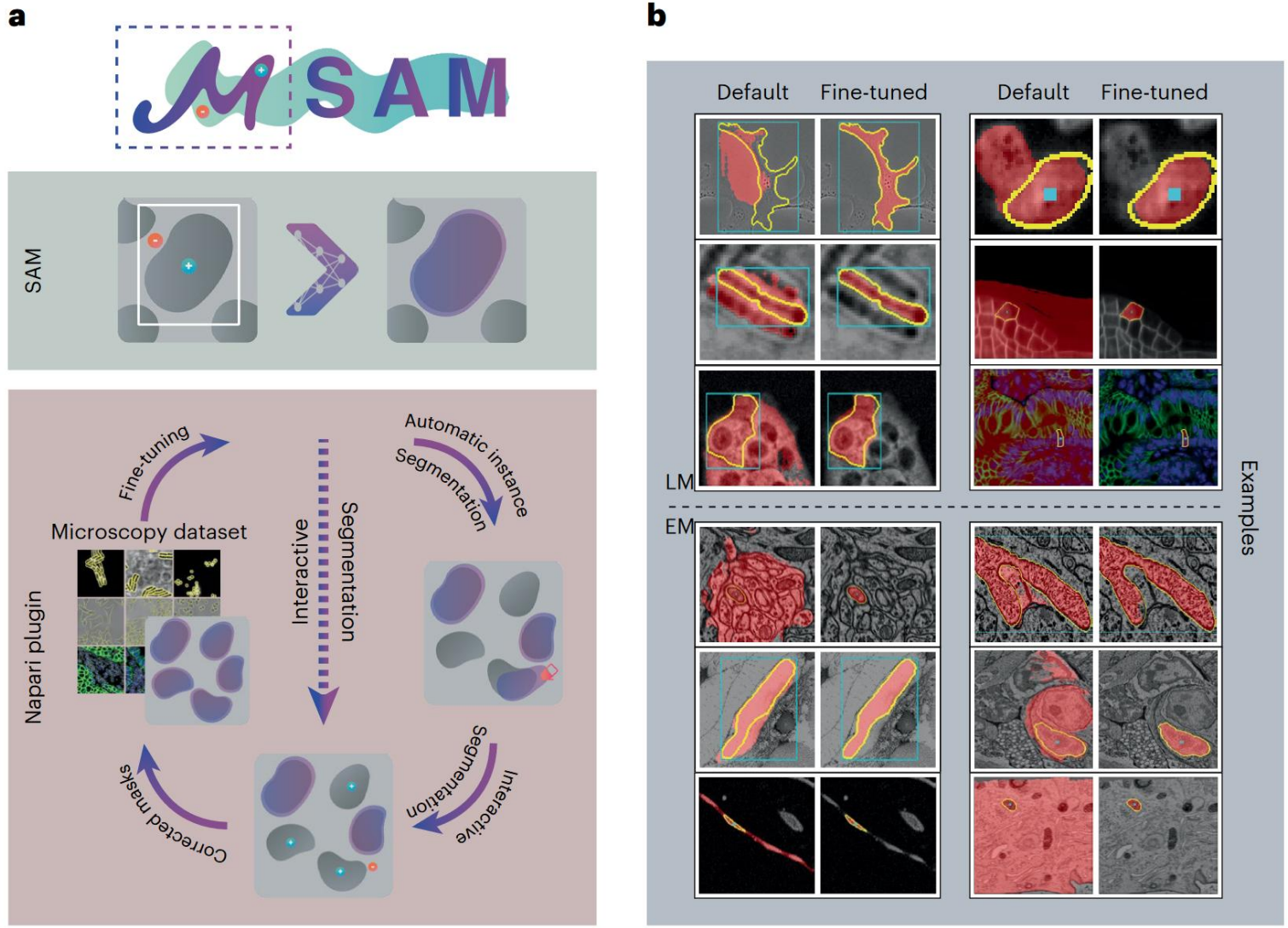


Fig. 1 | Overview of µSAM. a, We provide a napari plugin for segmenting multidimensional microscopy data. This tool uses SAM, including our improved models for LM and EM (see **b**). It supports automatic and interactive segmentation as well as model retraining on user data. The drawing sketches a complete workflow based on automatic segmentation, correction of the segmentation masks through interactive segmentation and model retraining based on the obtained annotations. Individual parts of this workflow can also be used on their own, for example, only interactive segmentation can be used as indicated by the dashed line. **b**, Improvement of segmentation quality due to our improved models for LM (top) and EM (bottom). Blue boxes or blue points show the user input, yellow outlines show the true object and red overlay depicts the model prediction.

Download and install git, then in a Miniforge **prompt**:

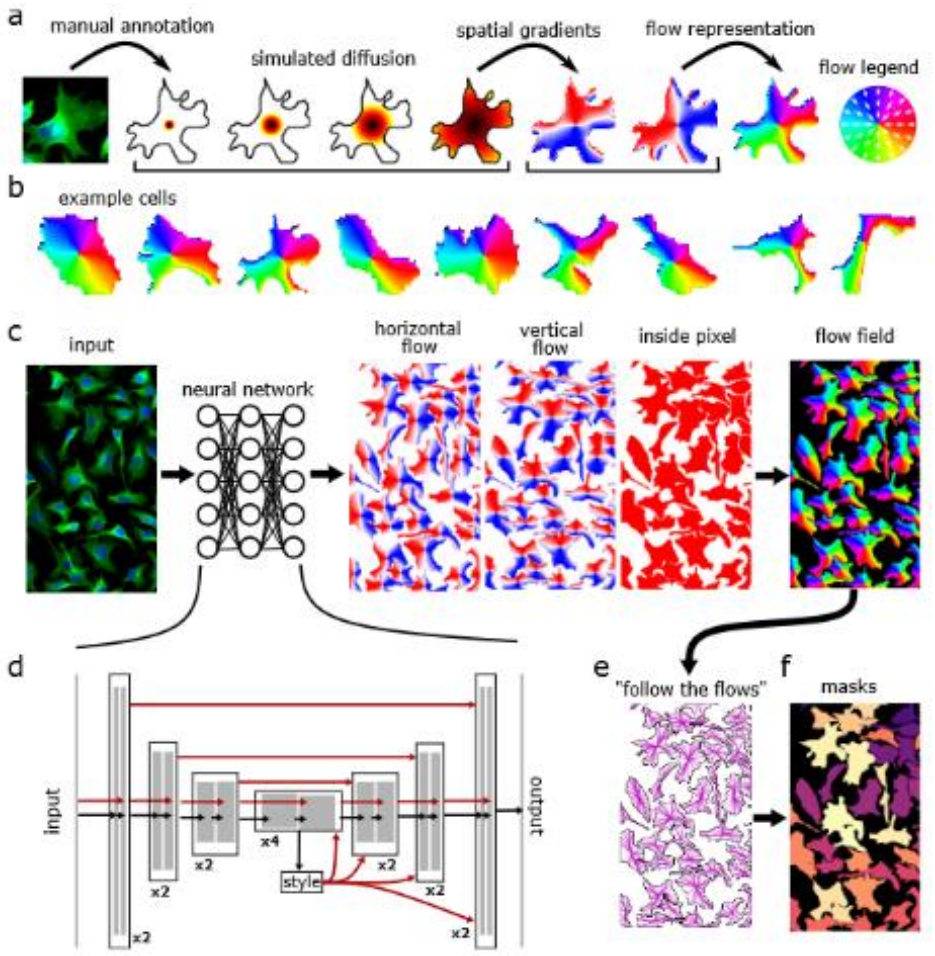
- mamba create -y -n microSAM_env python=3.11 // create a virtual environment called microSAM_Env
- mamba activate microSAM_env // activate the virtual environment NapariEnv
- mamba install -c ukoethe vigra // install vigra, a required package for µSAM
- mamba install python-elf // install python-elf, a required package for µSAM
- mamba install -c conda-forge torch_em // install torch_em, a required package for µSAM
- pip install napari[all] xxhash segment-anything torch torchvision zarr==2.13 numcodecs==0.15 git+https://github.com/ChaoningZhang/MobileSAM.git // install packages
- git clone https://github.com/computational-cell-analytics/micro-sam // download µSAM git
- cd micro-sam // go into folder downloaded from git
- pip install . // install last packages for µSAM

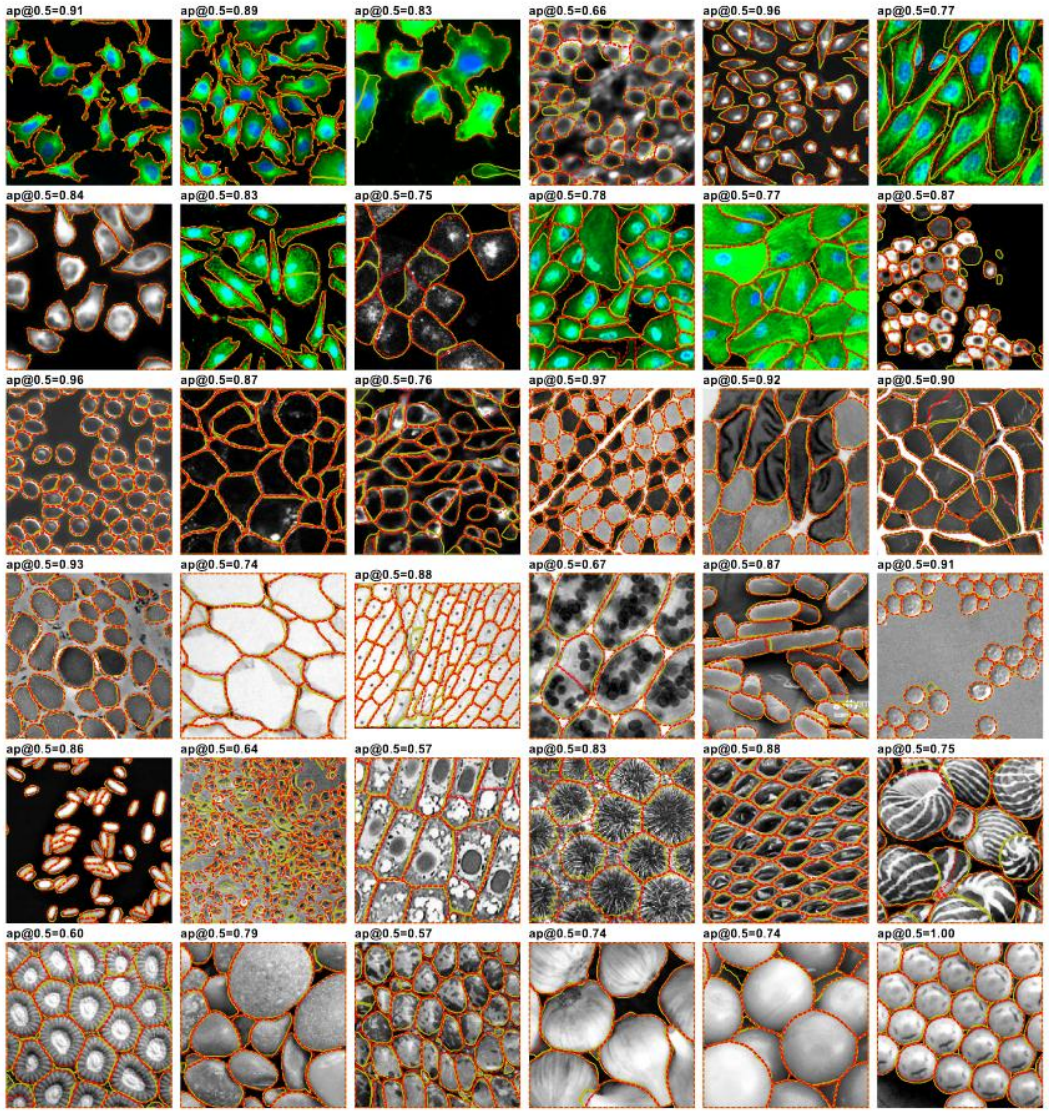
Download and install git, then in a Miniforge **prompt**:

- mamba create -y -n microSAM_env python=3.11 // create a virtual environment called microSAM_Env
- mamba activate microSAM_env // activate the virtual environment NapariEnv
- mamba install -c ukoethe vigra // install vigra, a required package for µSAM
- mamba install python-elf // install python-elf, a required package for µSAM
- mamba install -c conda-forge torch_em // install torch_em, a required package for µSAM
- pip install napari[all] xxhash segment-anything torch torchvision zarr==2.13 numcodecs==0.15 git+https://github.com/ChaoningZhang/MobileSAM.git // install packages
- git clone https://github.com/computational-cell-analytics/micro-sam // download µSAM git
- cd micro-sam // go into folder downloaded from git
- pip install . // install last packages for µSAM

- Use µSAM to annotate cells

CELLPOSE



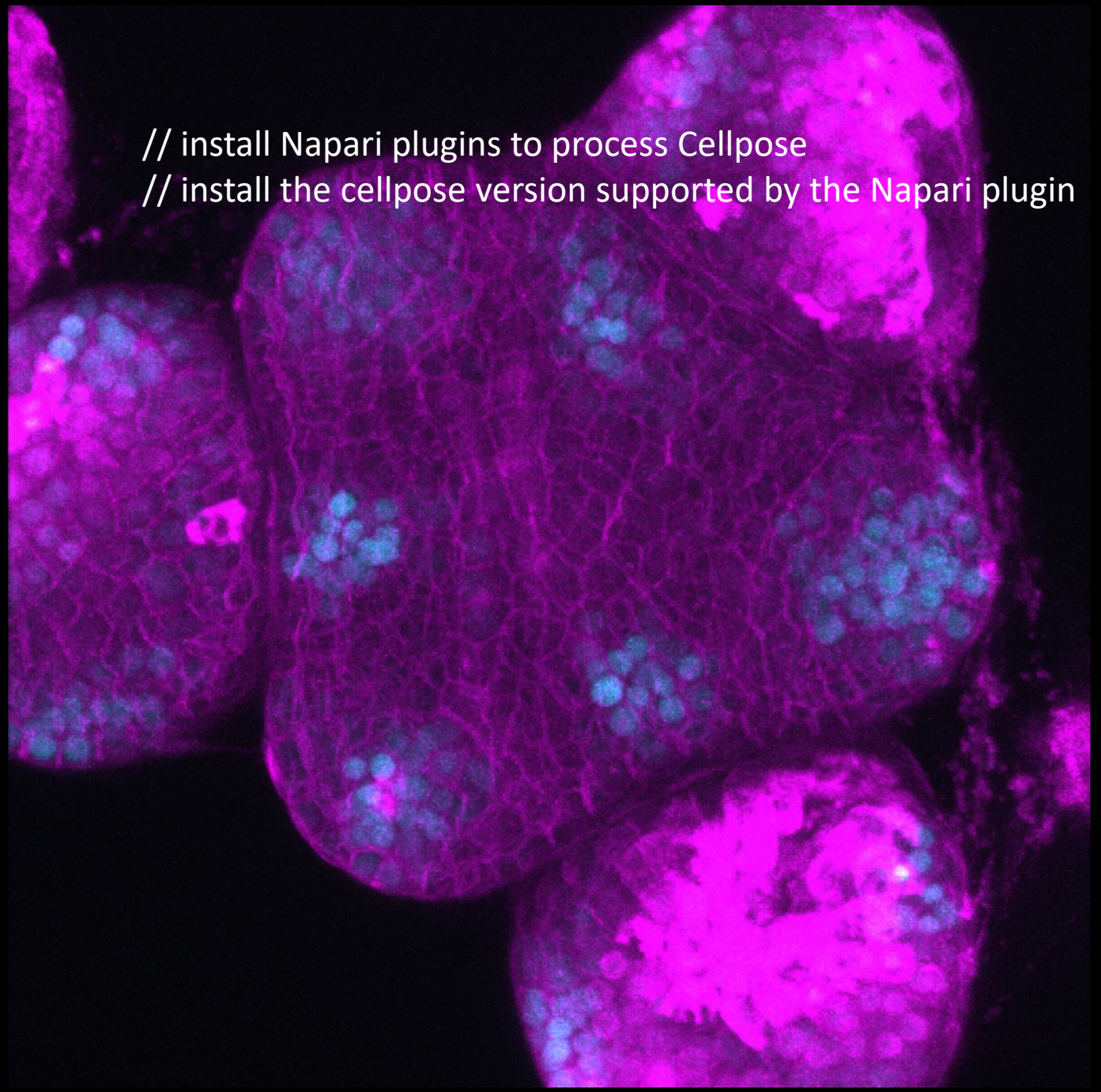


Stringer *et al.,* Cellpose: a generalist algorithm for cellular segmentation. *Nature Methods*, 2021

In a Miniforge **prompt**:

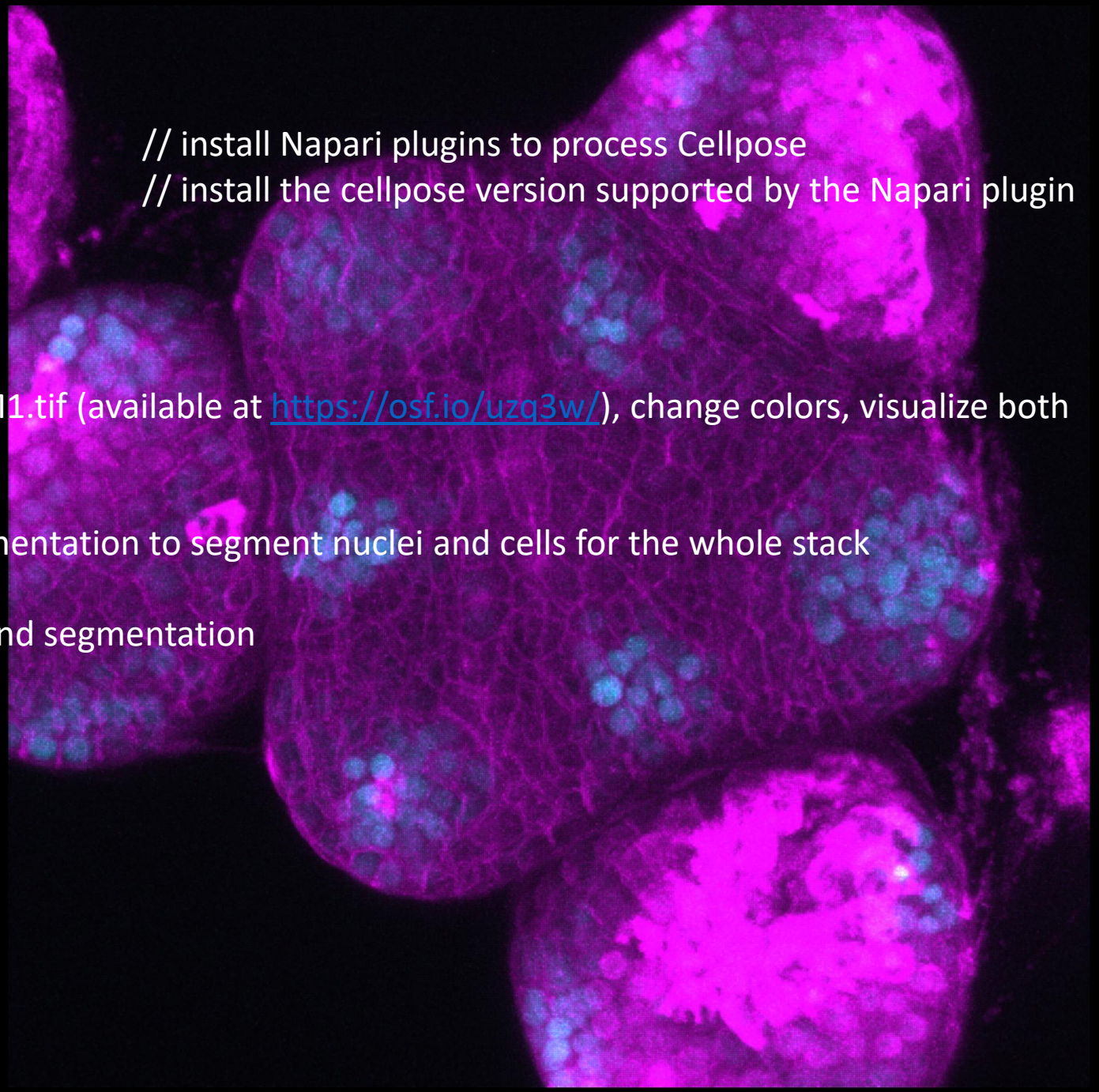
- pip install cellpose-napari
- pip install cellpose==3.0

// install Napari plugins to process Cellpose
// install the cellpose version supported by the Napari plugin



In a Miniforge **prompt**:

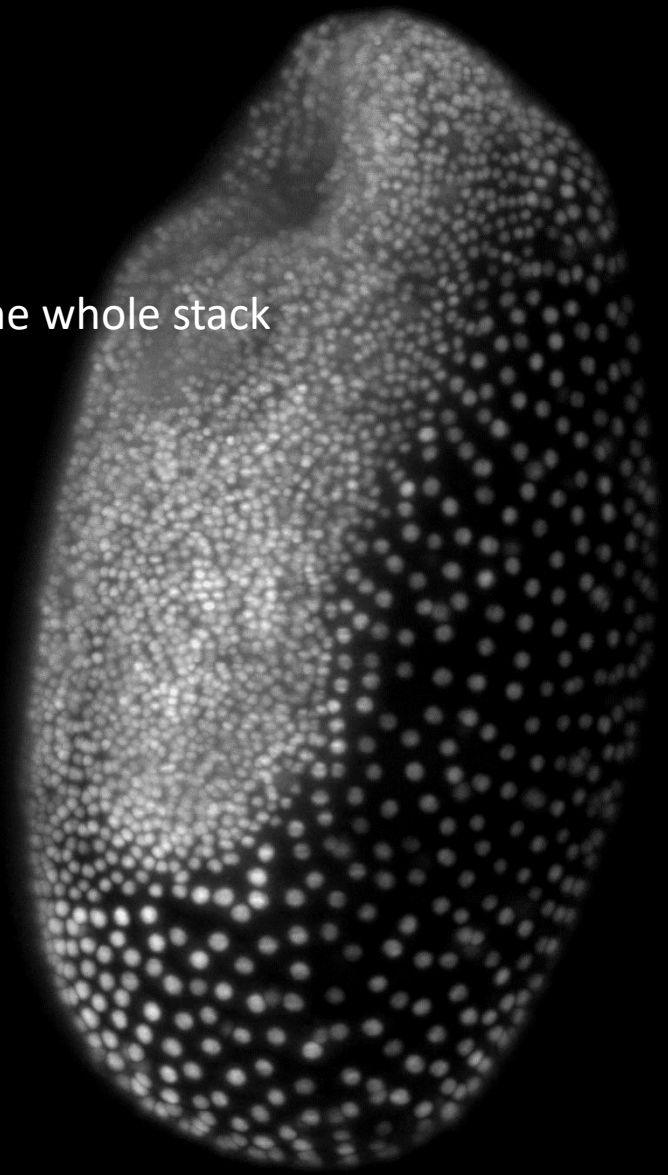
- pip install cellpose-napari // install Napari plugins to process Cellpose
- pip install cellpose==3.0 // install the cellpose version supported by the Napari plugin



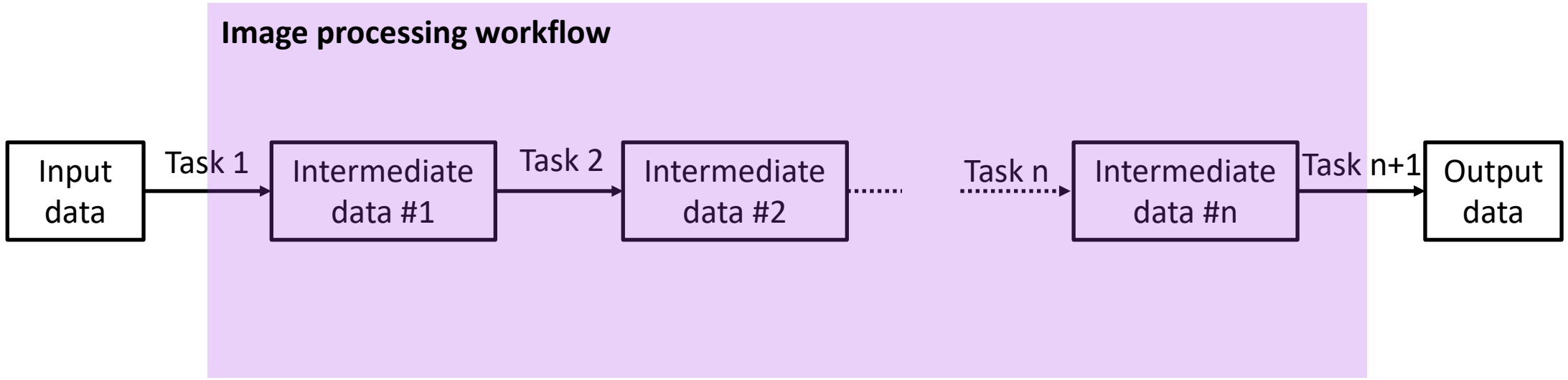
- Open MutX_DR5v2_6_4_M1.tif (available at https://osf.io/uzq3w/), change colors, visualize both channels
- Parameterize cellpose segmentation to segment nuclei and cells for the whole stack
- Make a video with image and segmentation

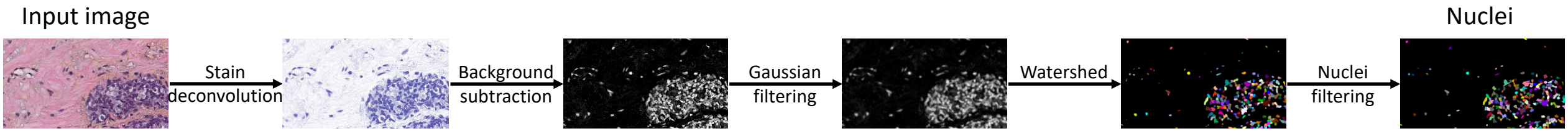
- Open lund1051_resampled.tif (available at https://git.mpi-cbg.de/rhaase/clij2_example_data/blob/master/lund1051_resampled.tif)

- Open lund1051_resampled_crop.tif to tune Cellpose parameters and segment the whole stack

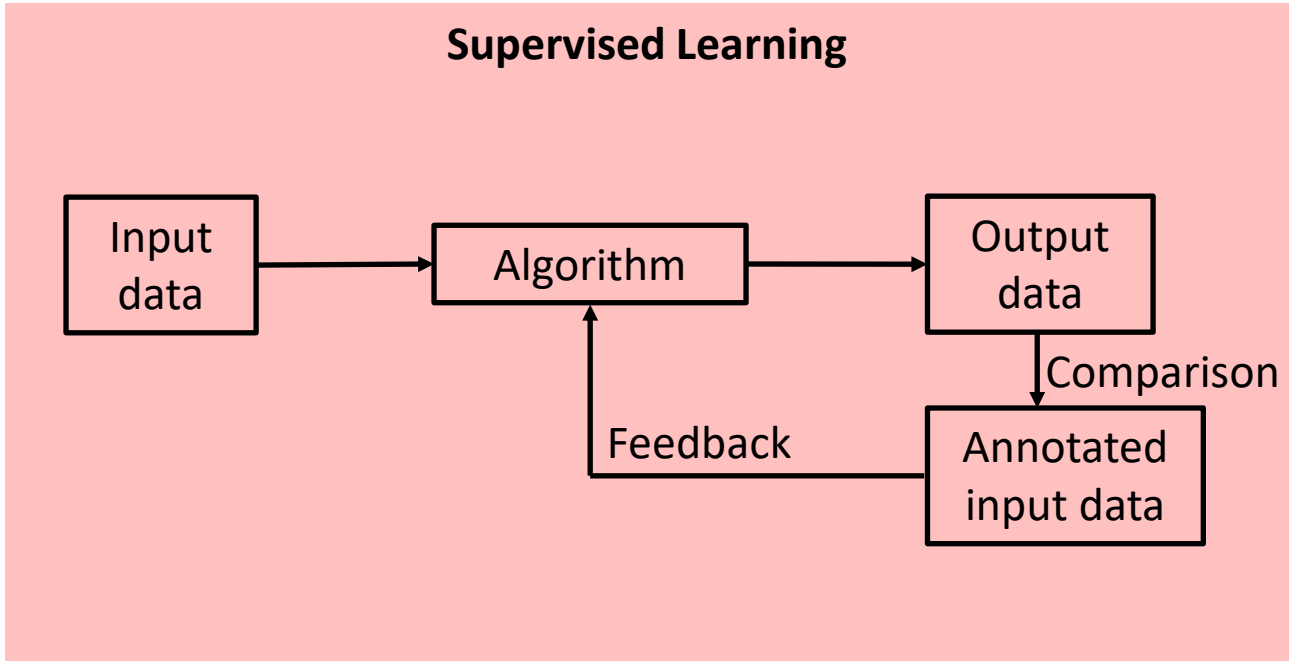


EXPLICIT PROGRAMMING

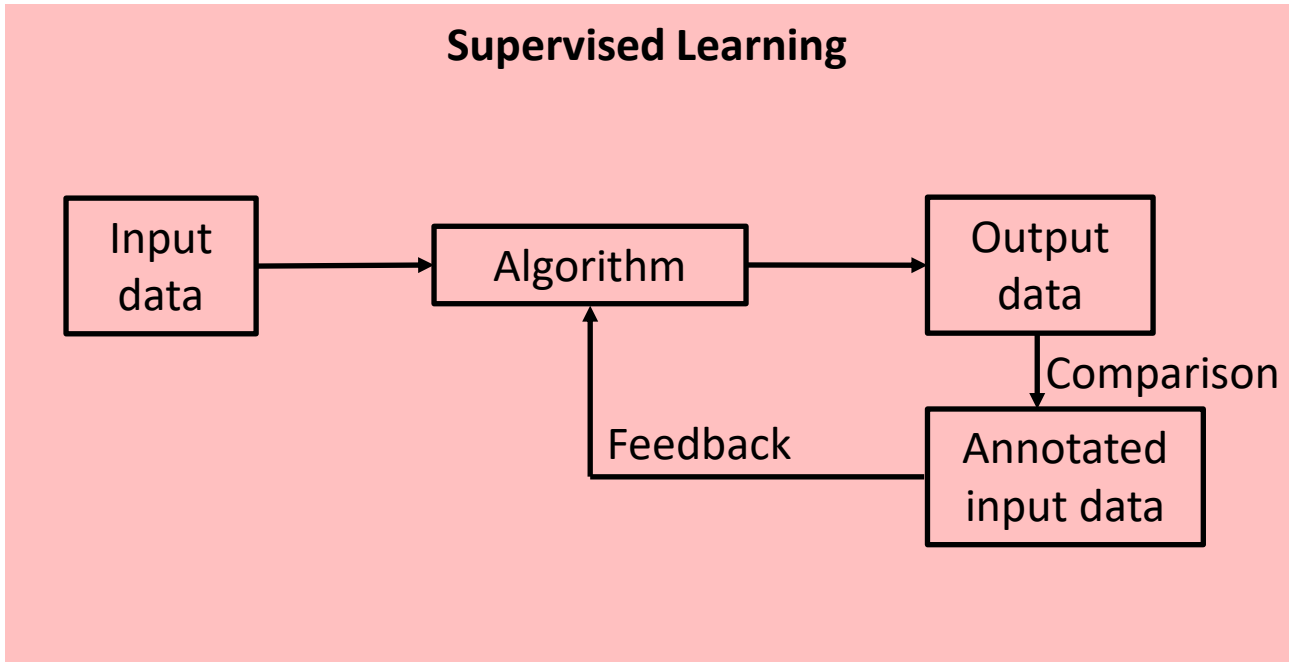


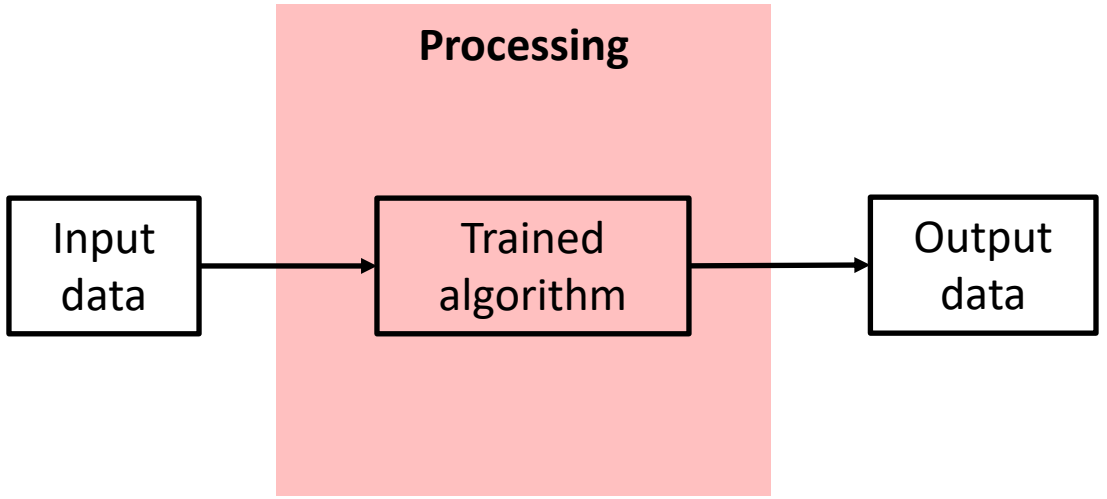


Supervised Machine Learning

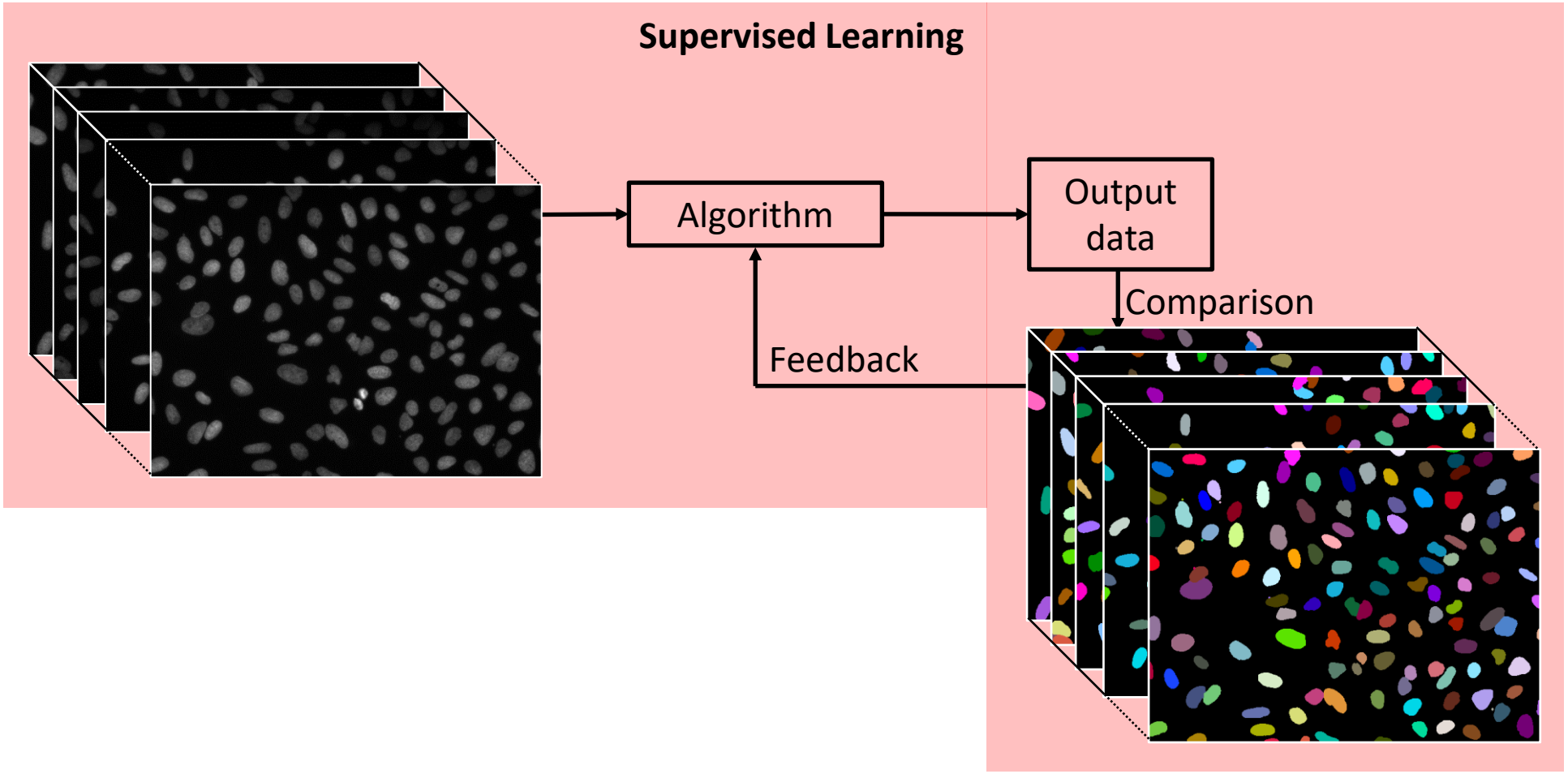


Supervised Machine Learning

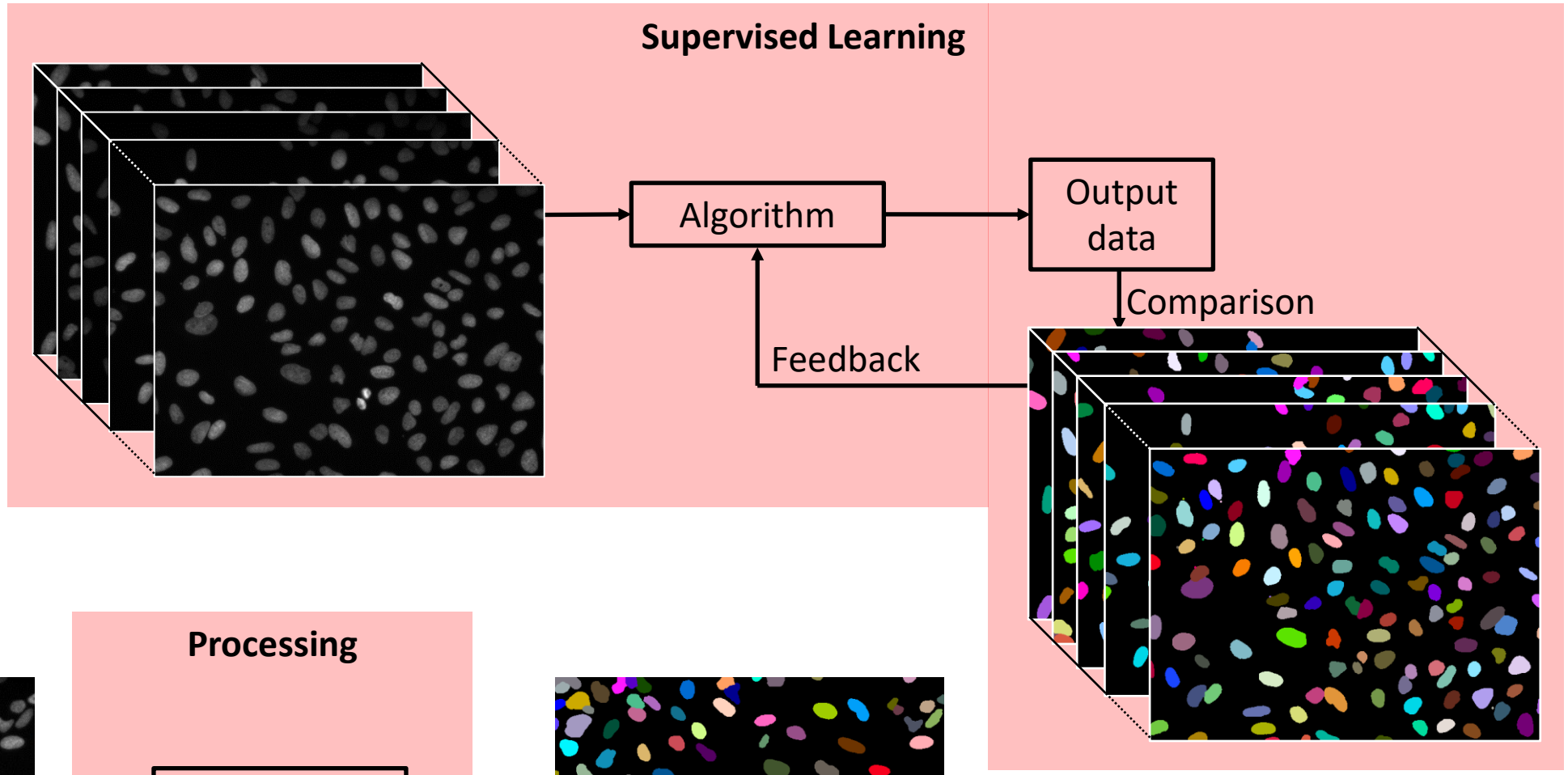


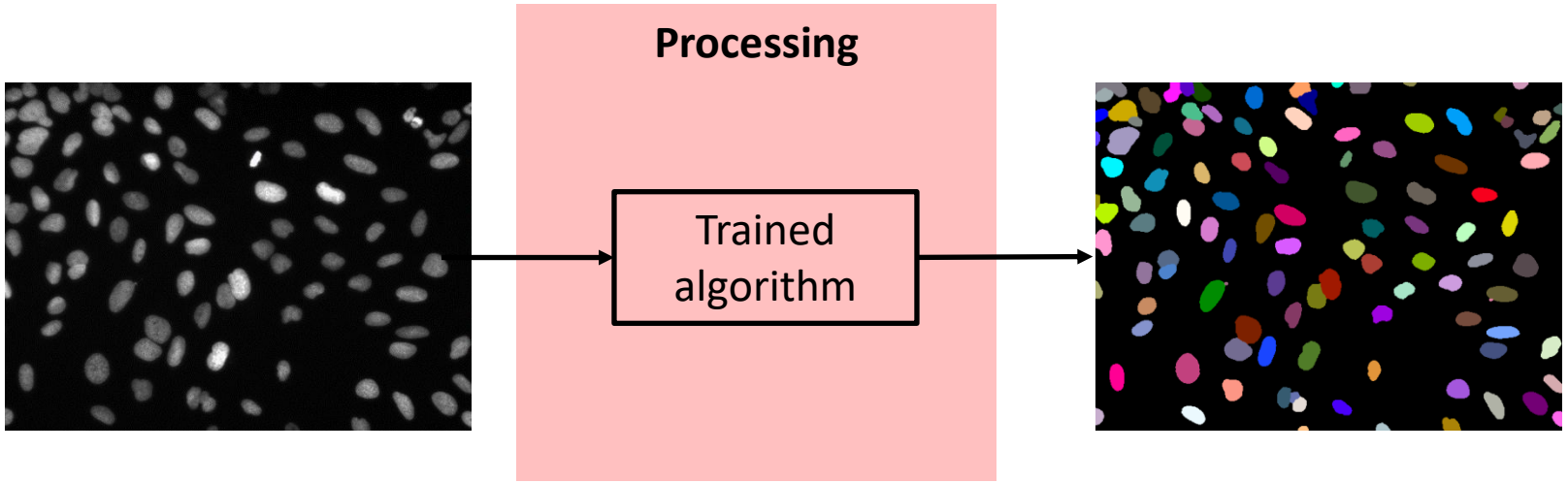


Supervised Machine Learning



Supervised Machine Learning



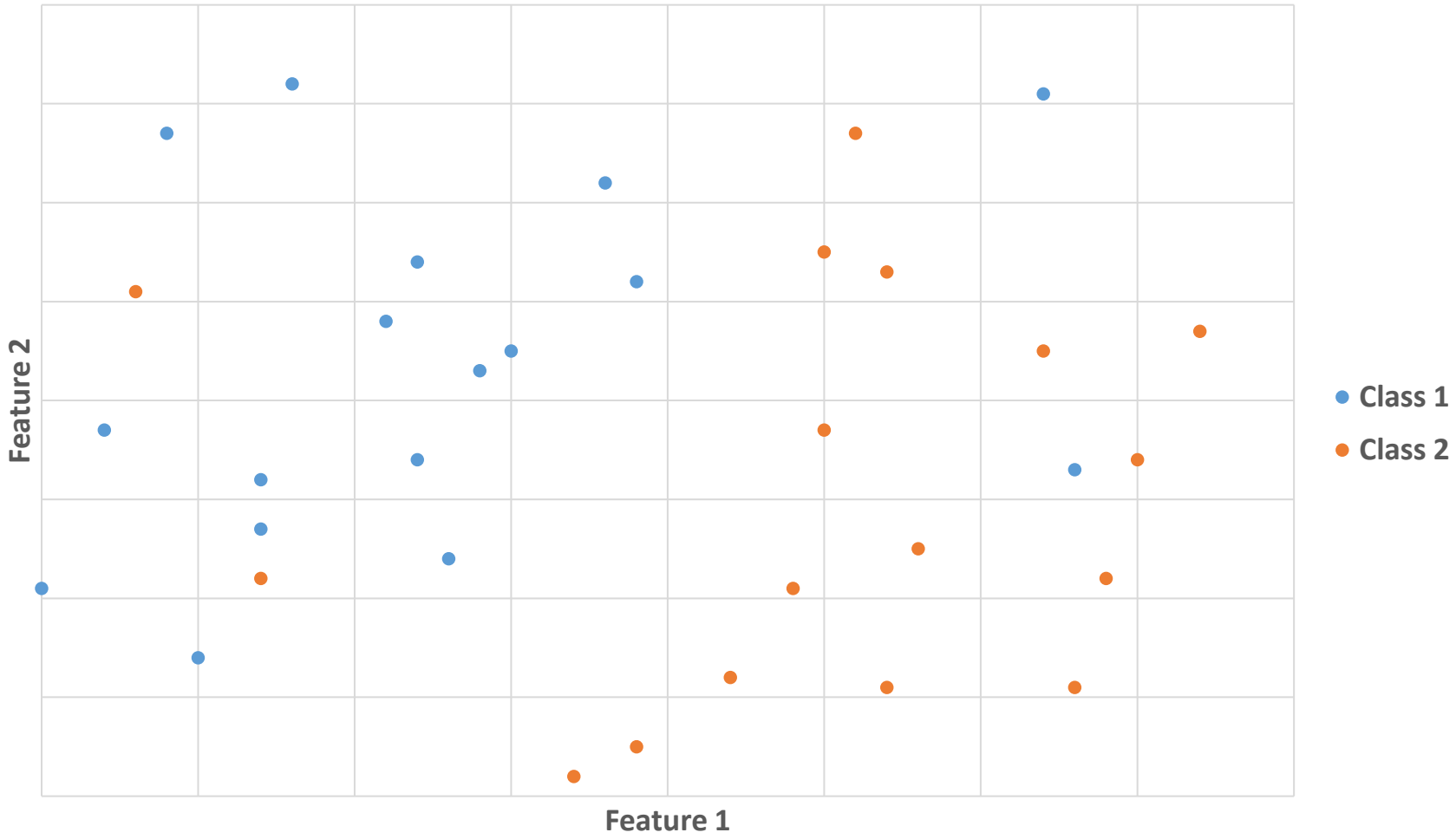


Shallow Machine Learning for Pixel Classification

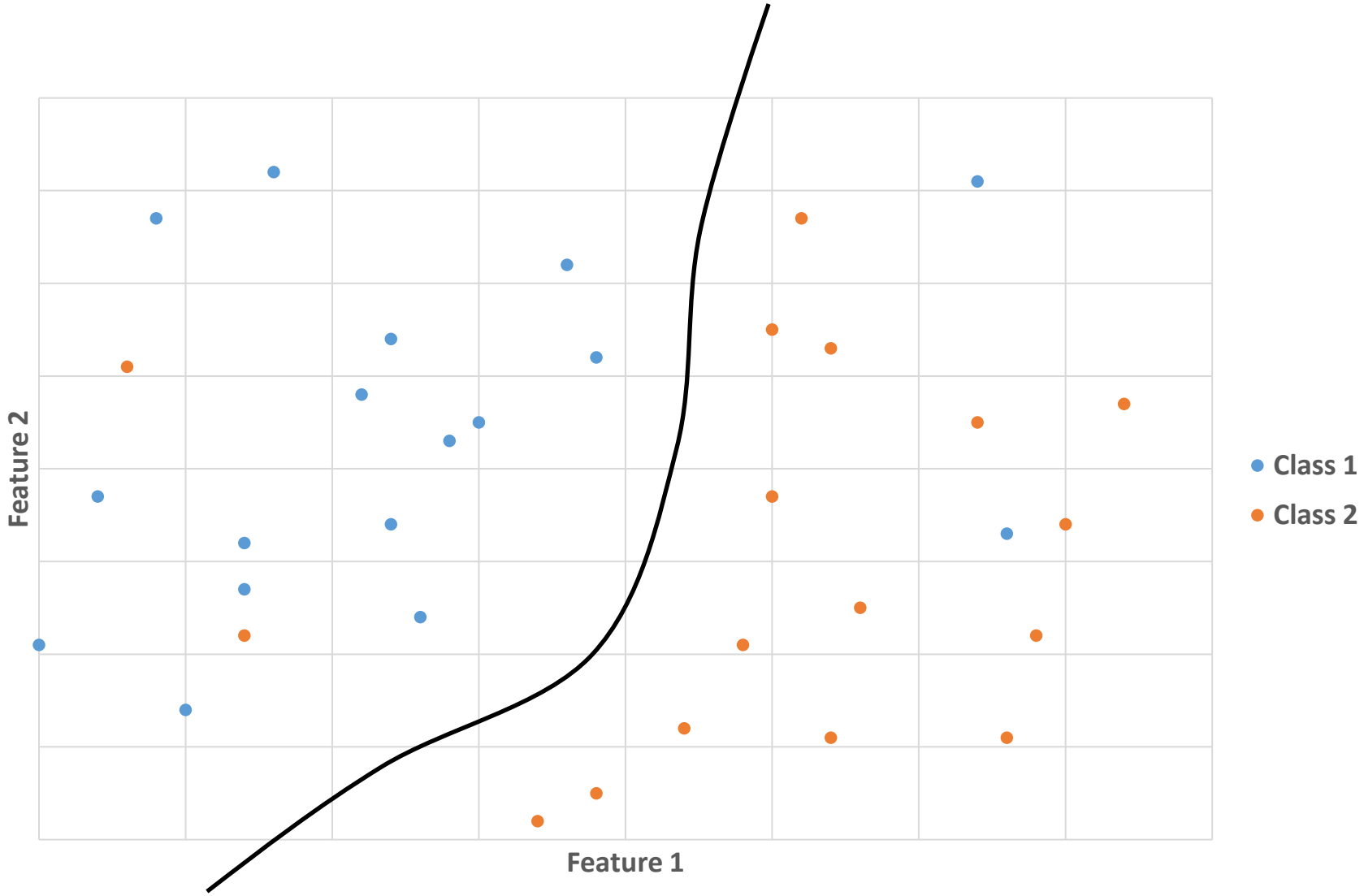
Supervised classification:

- **Examples** of classes are **manually** defined by the user
- A **classifier** is **trained** by using **defined features** with these examples
- Data is then **automatically classified** by using the trained classifier

Shallow Machine Learning for Pixel Classification

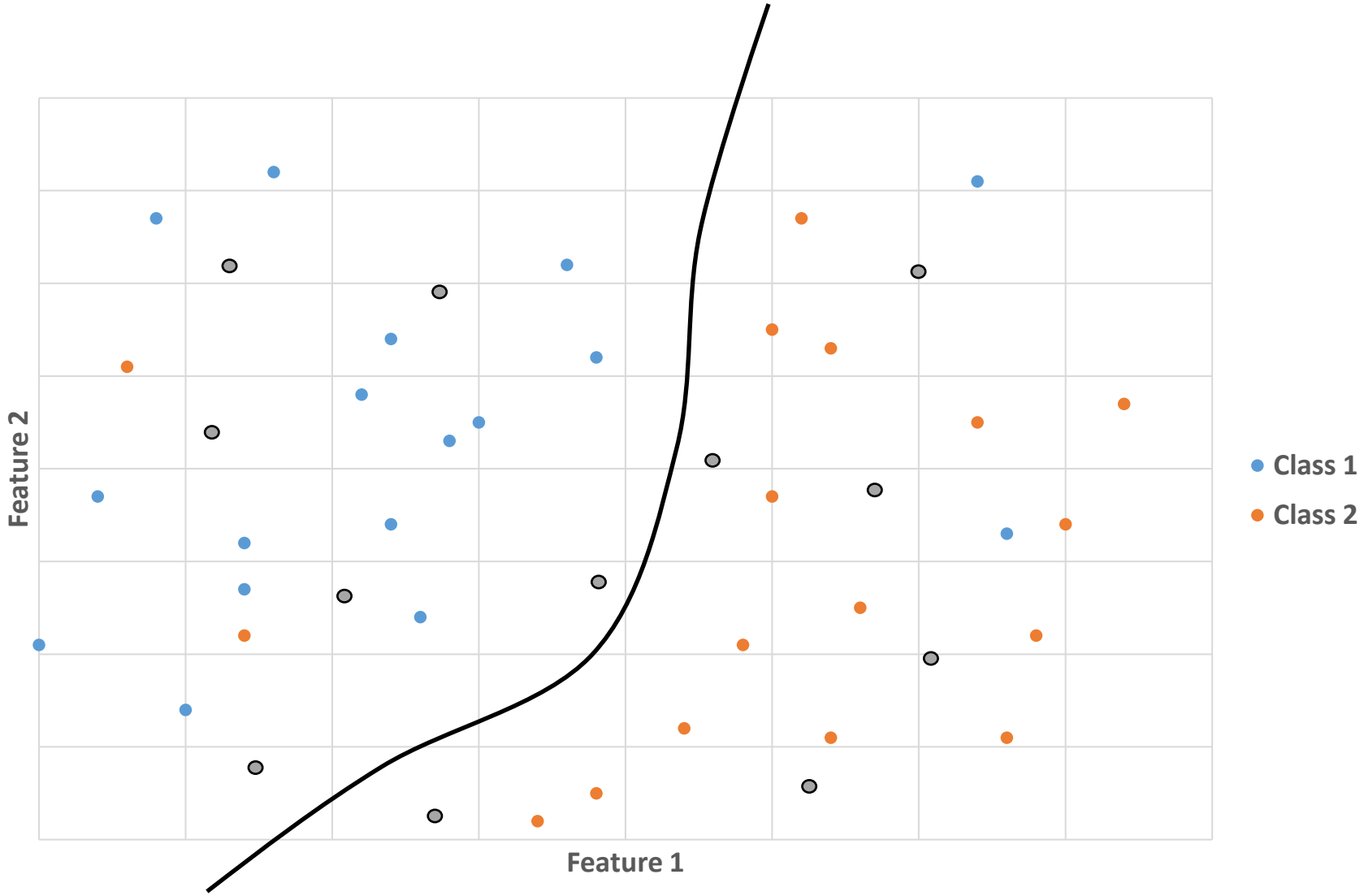


Shallow Machine Learning for Pixel Classification



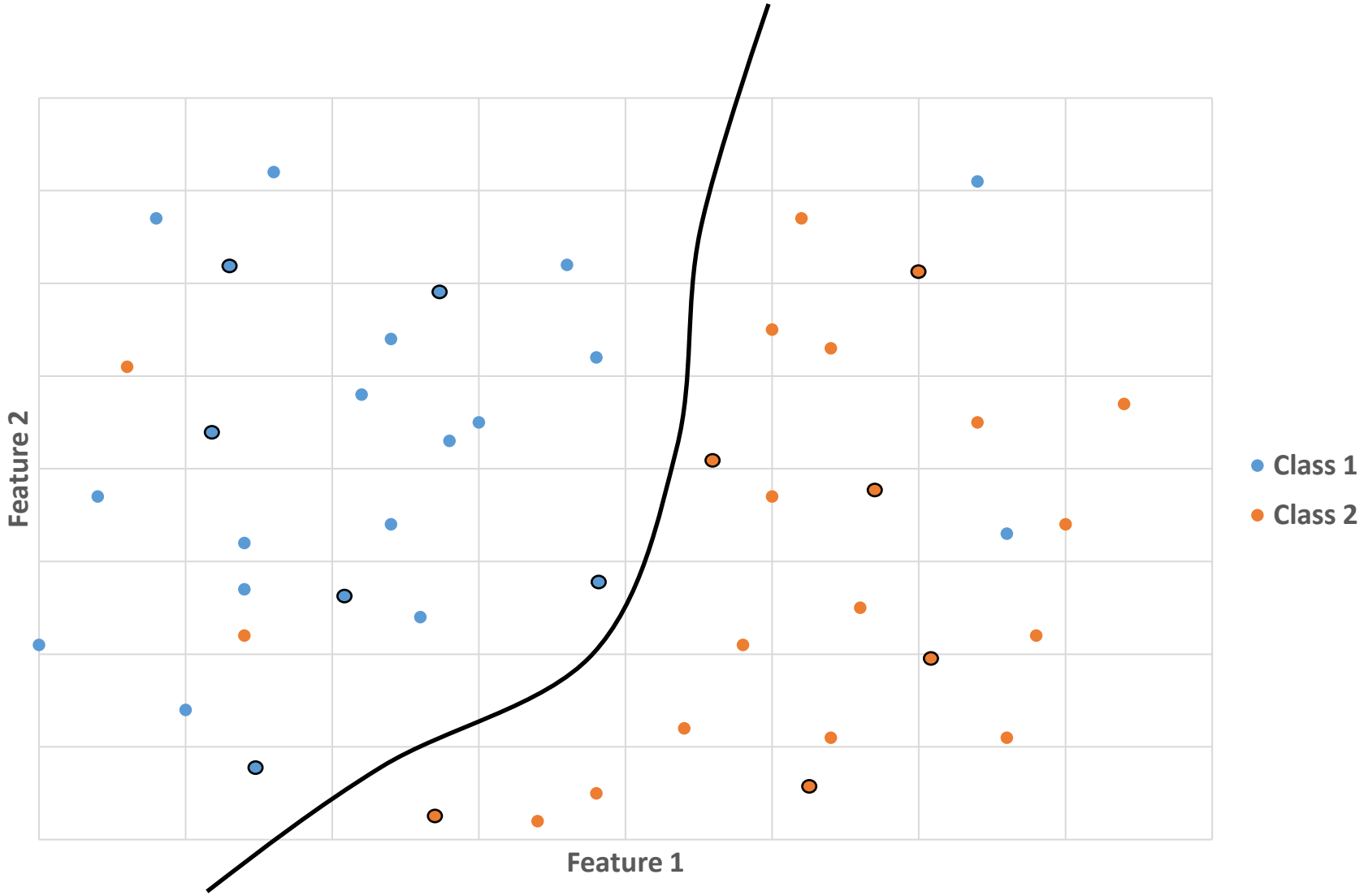
Training a classifier consists in **estimating** the **"best" separation** between classes

Shallow Machine Learning for Pixel Classification



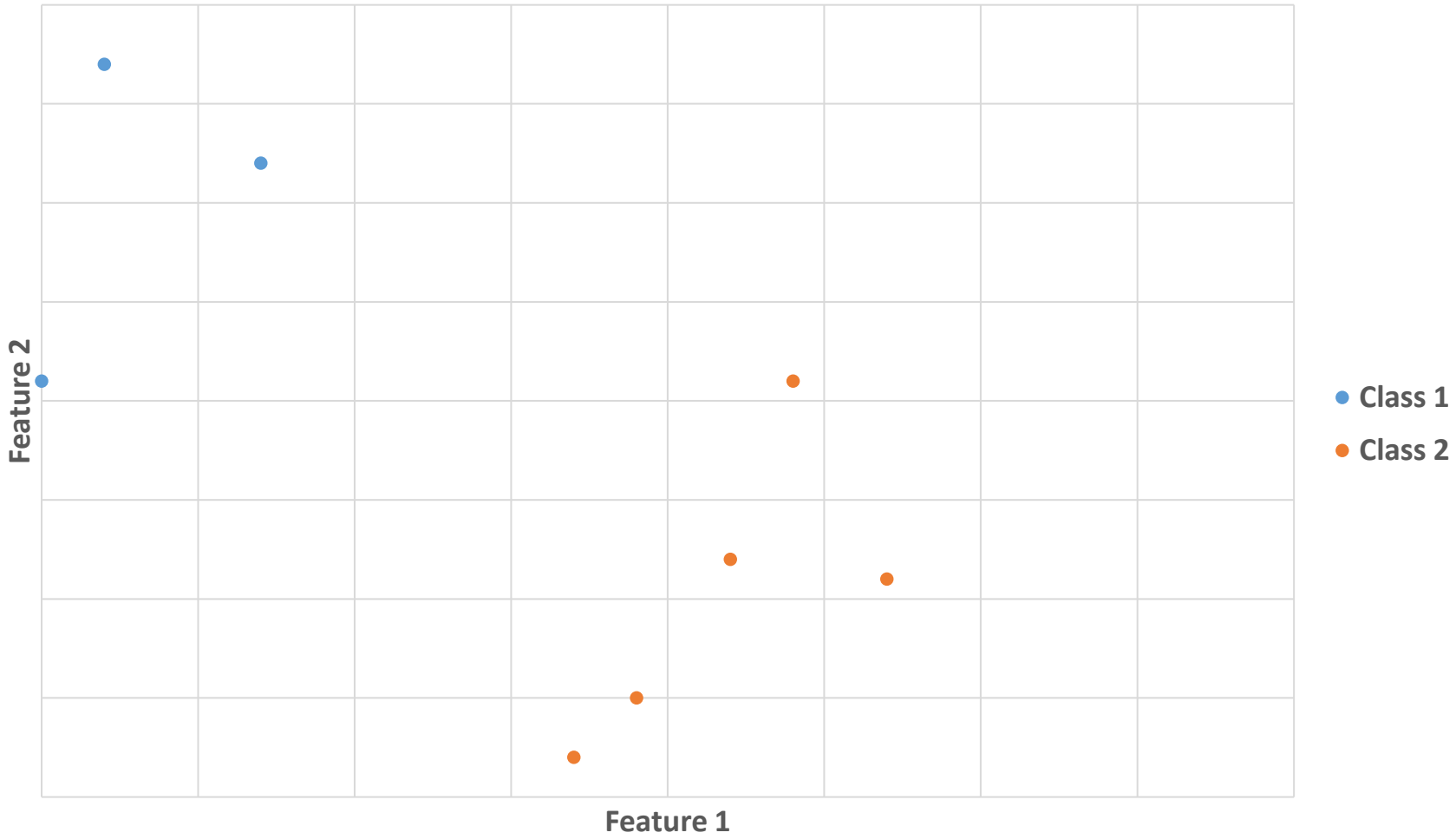
The **estimated class** for new data will be given by the **trained classifier**

Shallow Machine Learning for Pixel Classification



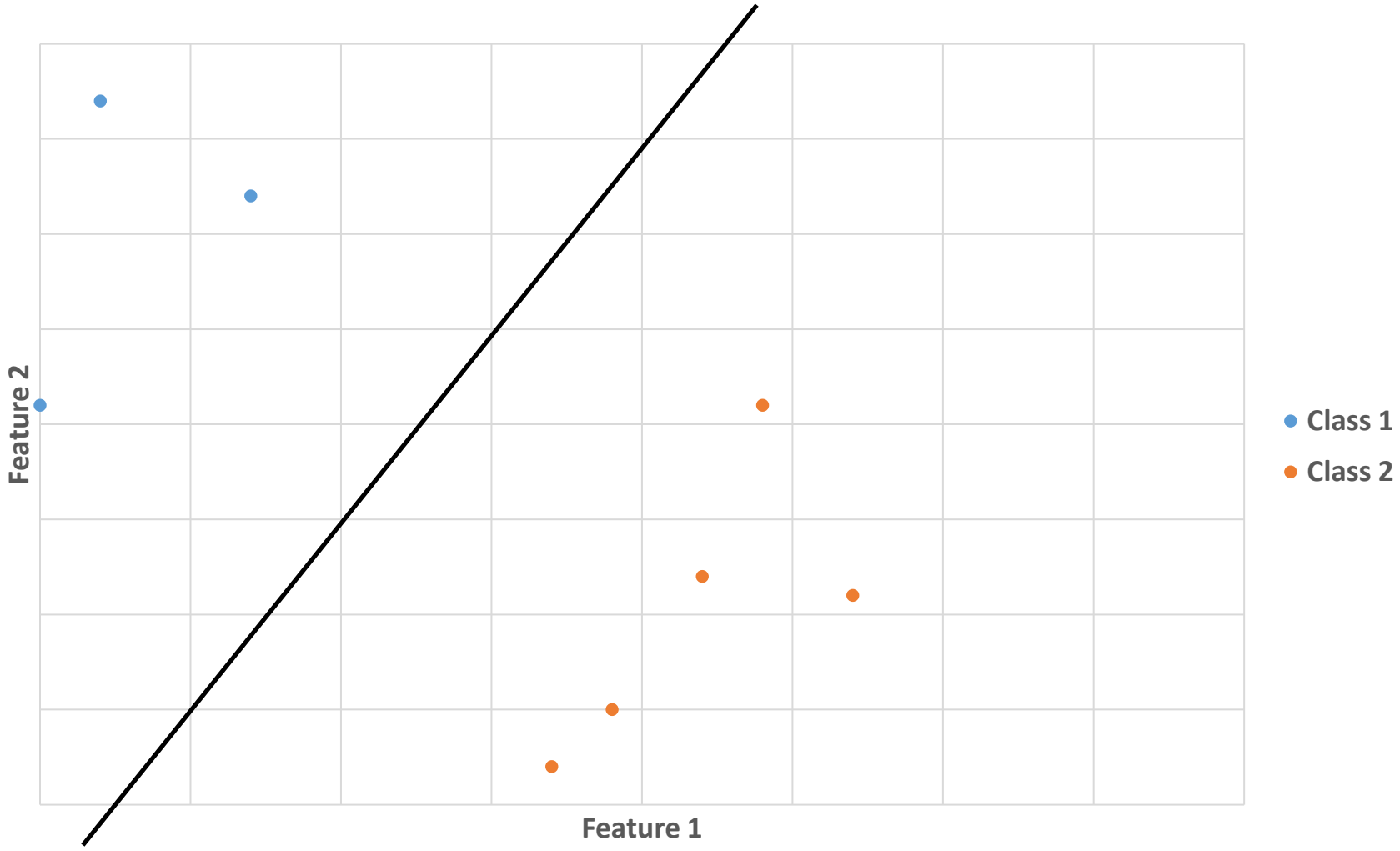
The **estimated class** for new data will be given by the **trained classifier**

Shallow Machine Learning for Pixel Classification



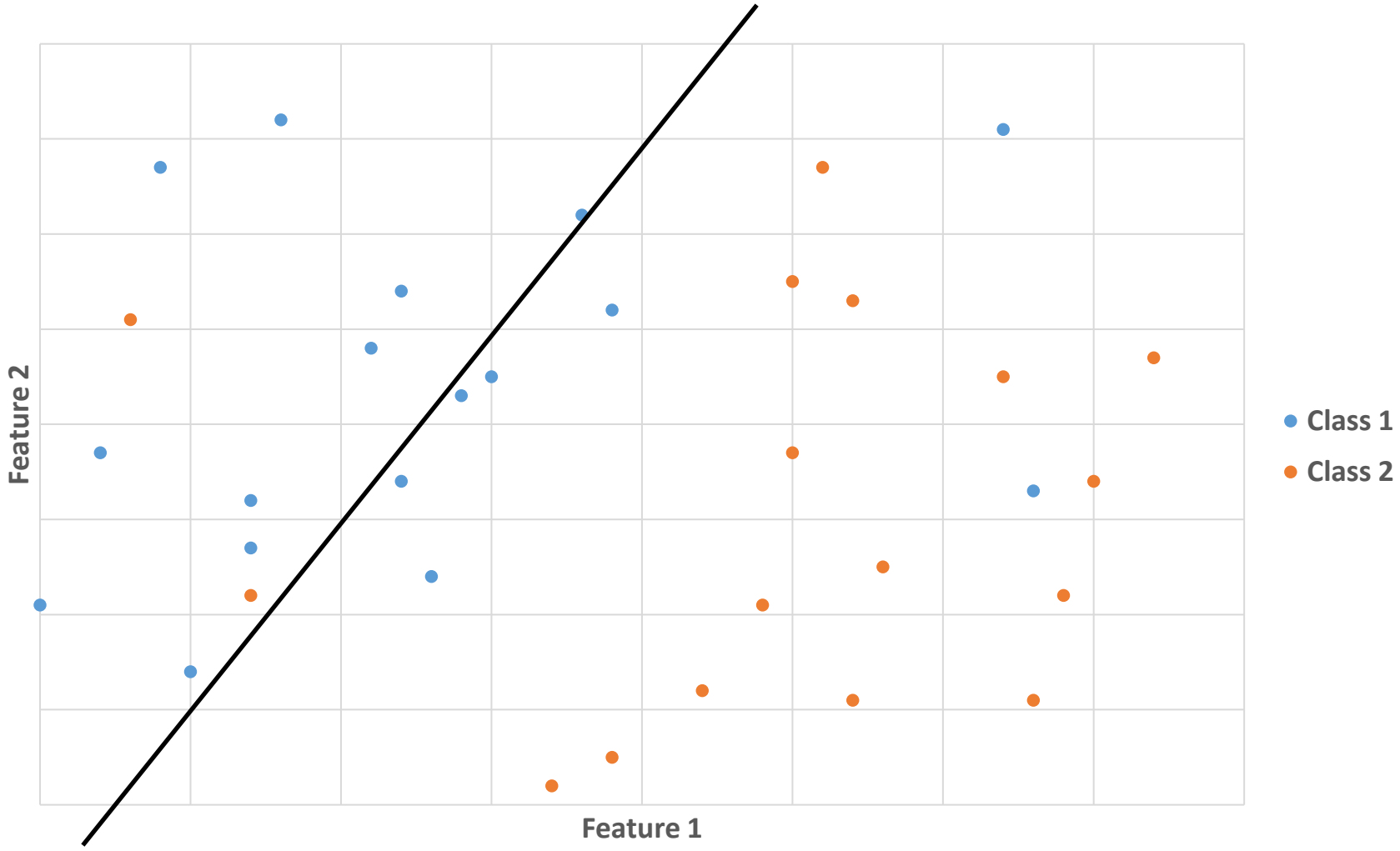
Training will be easier if **enough** and **representative** data is used

Shallow Machine Learning for Pixel Classification



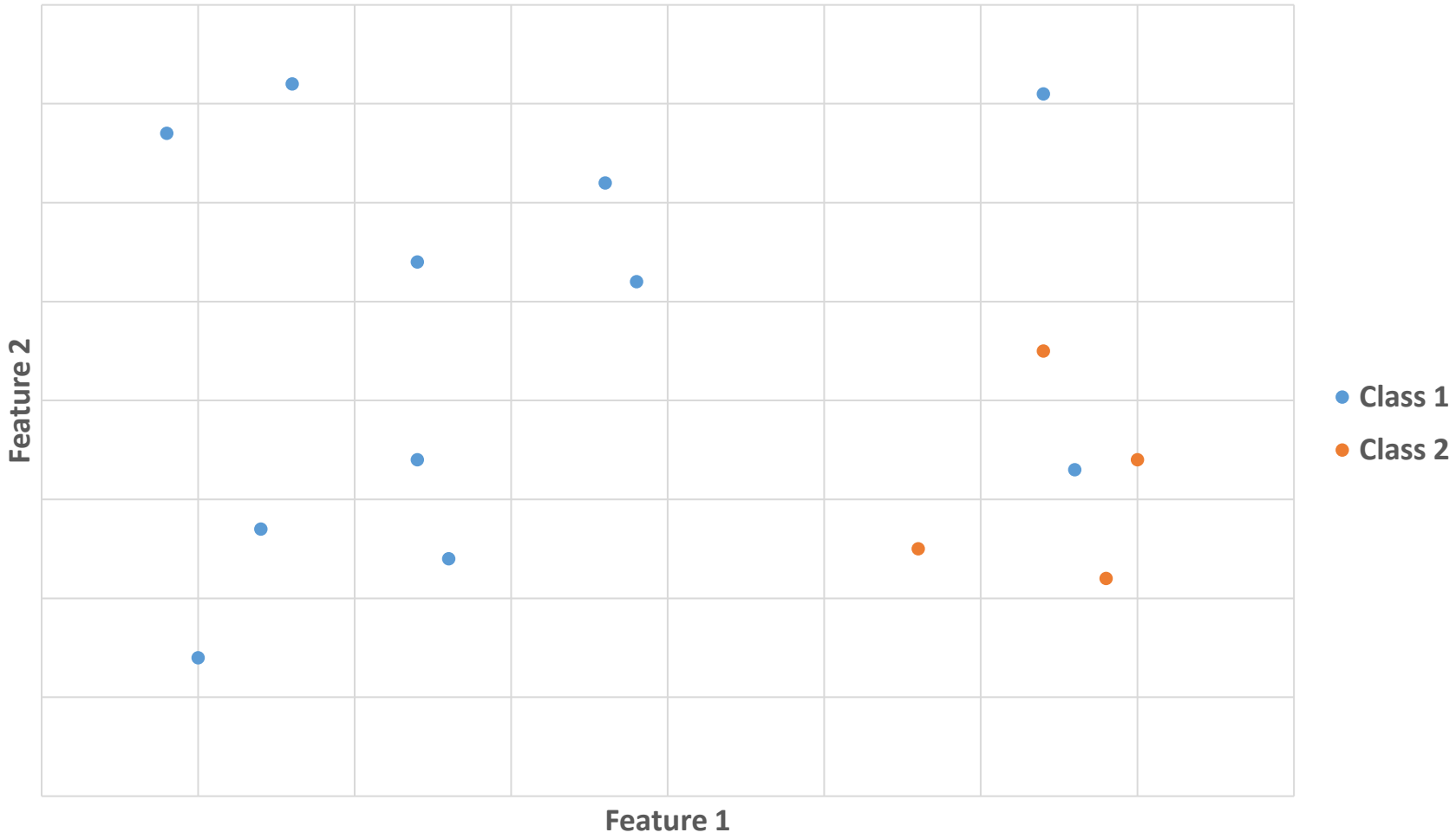
Training will be easier if **enough** and **representative** data is used

Shallow Machine Learning for Pixel Classification



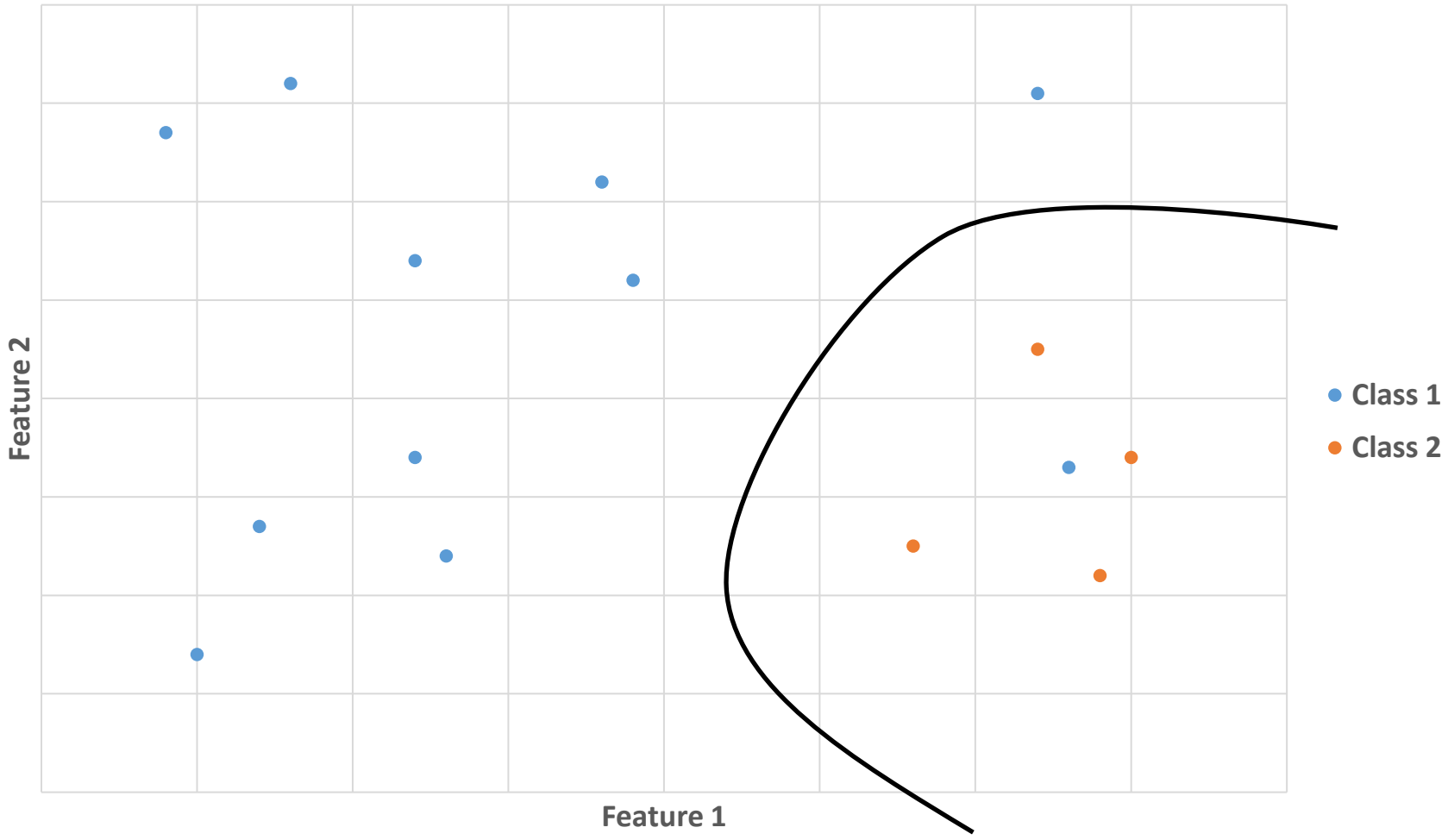
Training will be easier if **enough** and **representative** data is used

Shallow Machine Learning for Pixel Classification



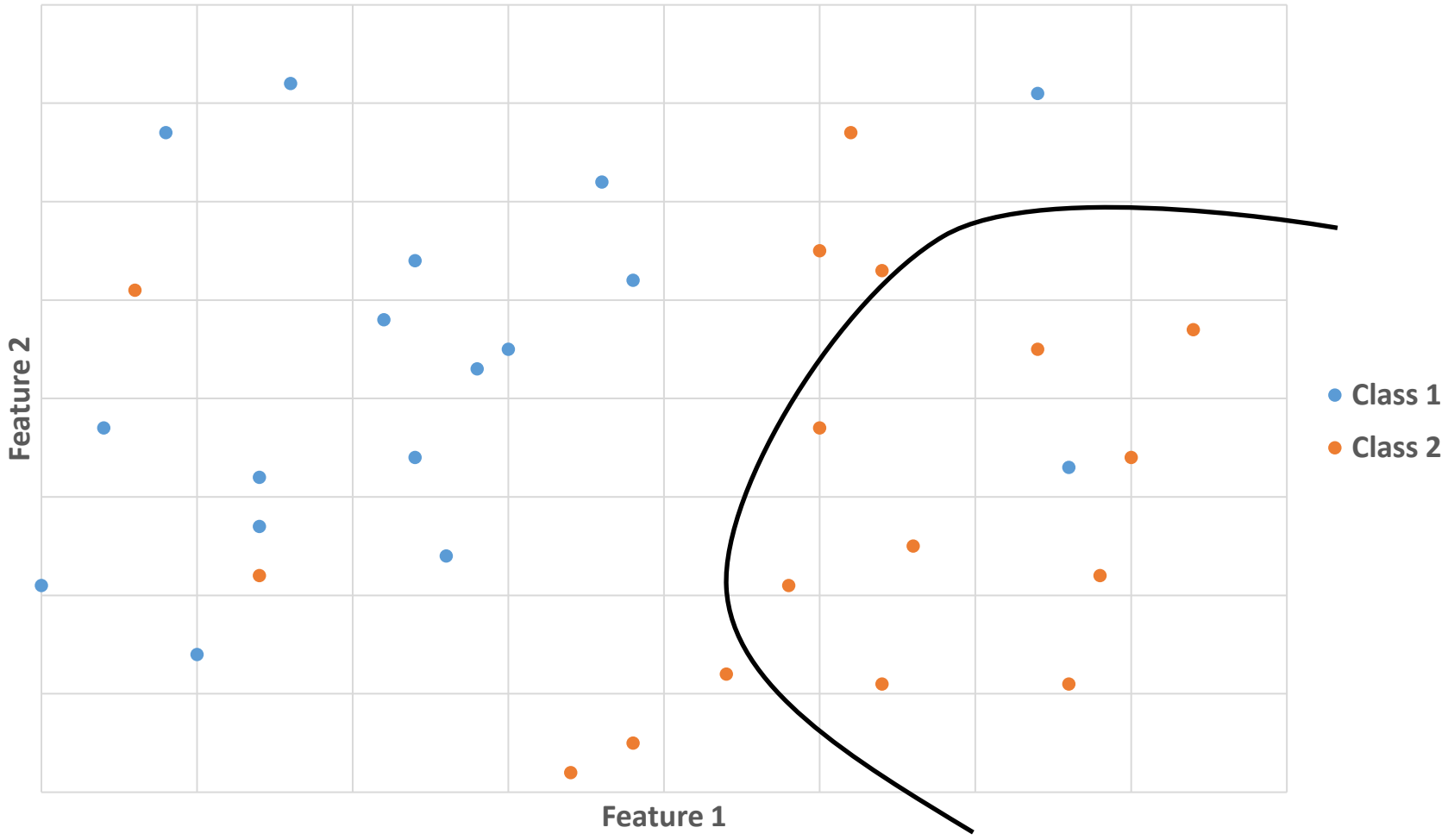
Class imbalance makes the training **more difficult**

Shallow Machine Learning for Pixel Classification



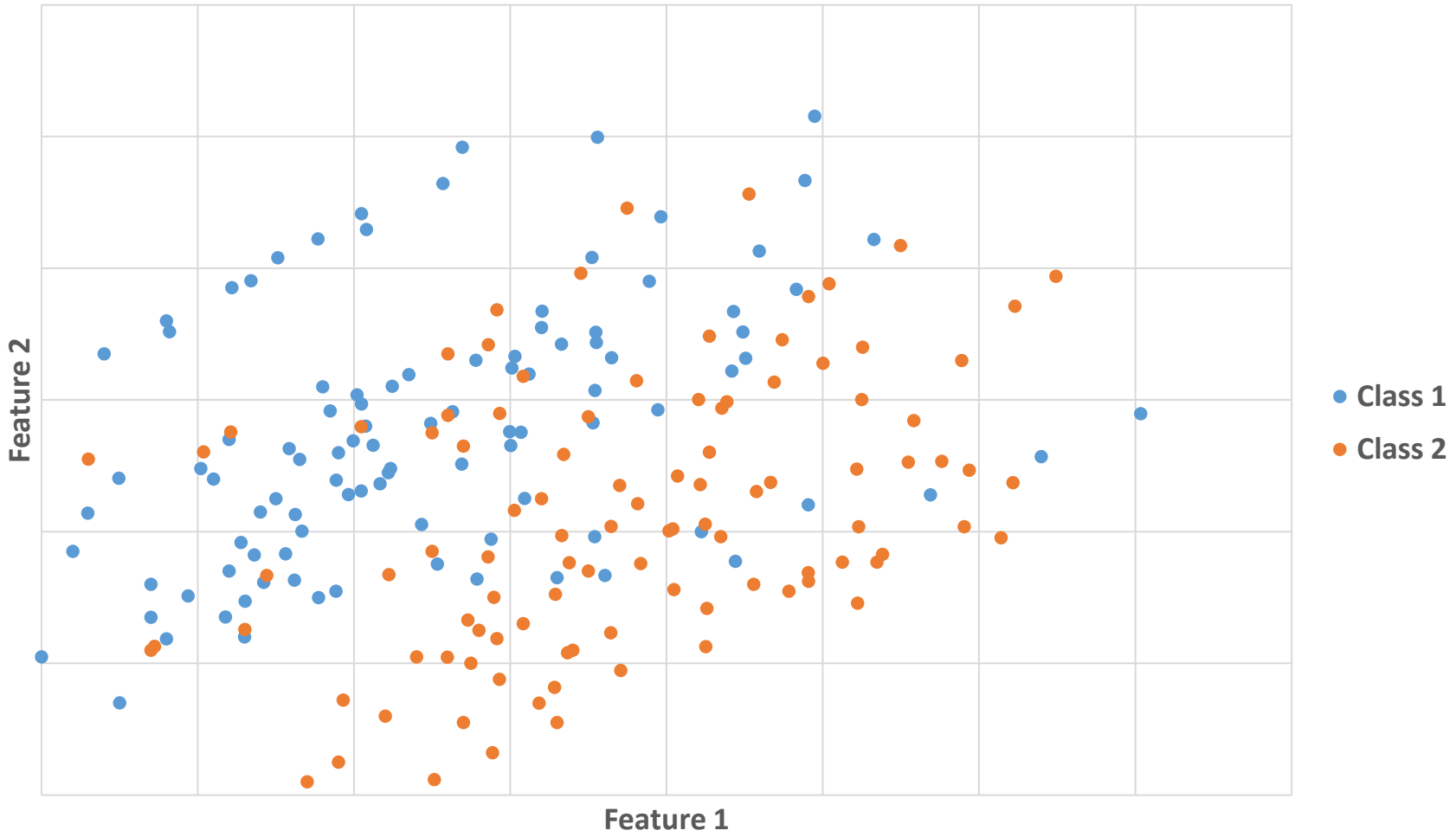
Class imbalance makes the training **more difficult**

Shallow Machine Learning for Pixel Classification



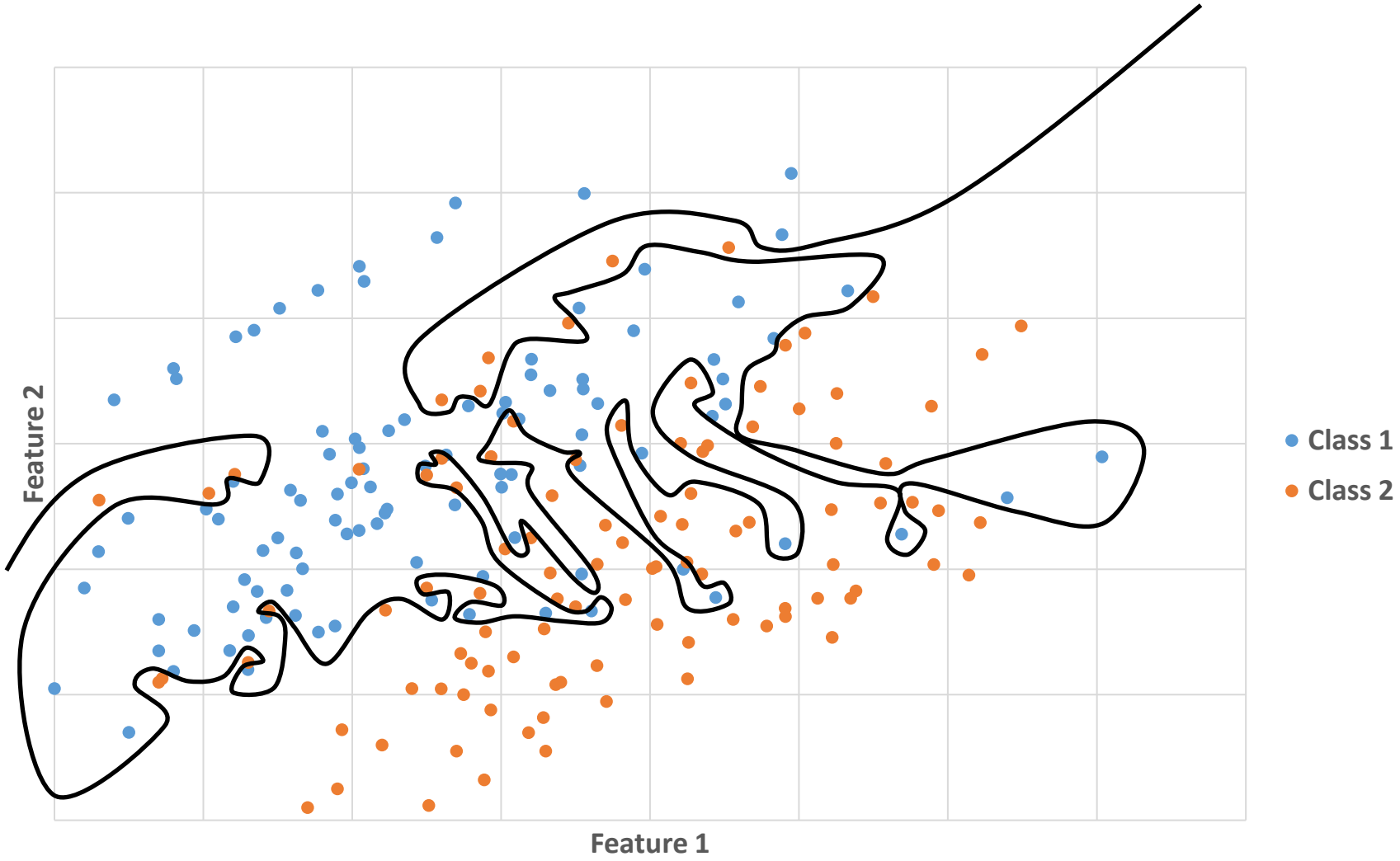
Class imbalance makes the training **more difficult**

Shallow Machine Learning for Pixel Classification



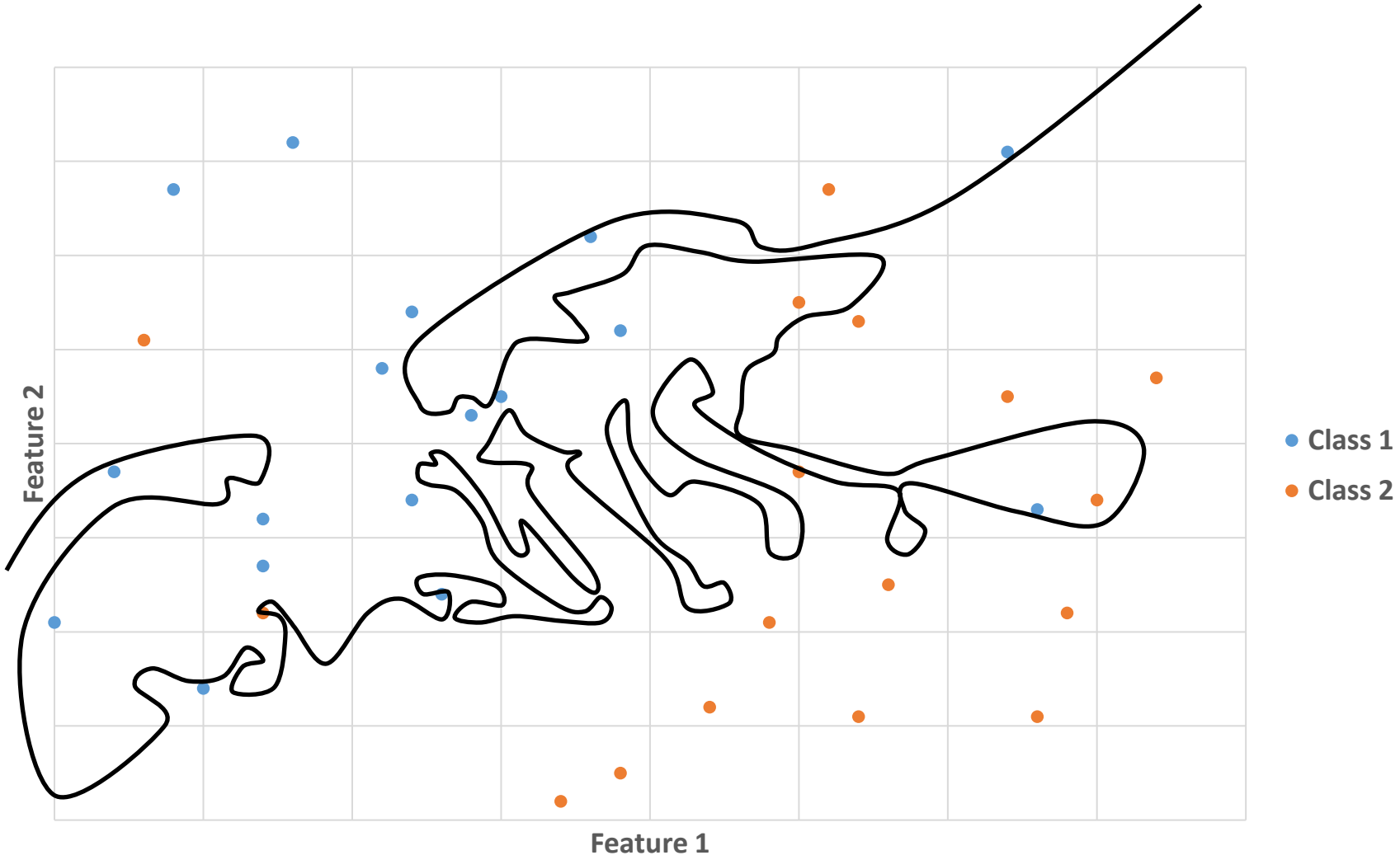
Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

Shallow Machine Learning for Pixel Classification



Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

Shallow Machine Learning for Pixel Classification



Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

Shallow Machine Learning for Pixel Classification

- Select **image features appropriate** to the classification problem
- Manually annotate regions/objects that are **representative** of what is seen in images
- Use **V** tool for annotations to **avoid over-representation**
- Define roughly the **same amount** of annotations for **each class**
- **Do not** manually annotate an **entire region of slide** to avoid over-fitting

nature methods

Explore content | About the journal | Publish with us | Subscribe

nature › nature methods › correspondence › article

Correspondence | Published: 18 November 2019

CLIJ: GPU-accelerated image processing for everyone

Robert Haase, Loic A. Royer, Peter Steinbach, Deborah Schmidt, Alexandr Dibrov, Uwe Schmidt, Martin Weigert, Nicola Maghelli, Pavel Tomancak, Florian Jug & Eugene W. Myers

Nature Methods **17**, 5–6 (2020) | Cite this article

5239 Accesses | **21** Citations | **95** Altmetric | Metrics

To the editor – Modern microscopy generates staggering amounts of multidimensional image data that place increasing demands on processing flexibility and efficiency. One way to speed up image processing is to exploit the parallel processing capabilities of graphics processing units (GPU).

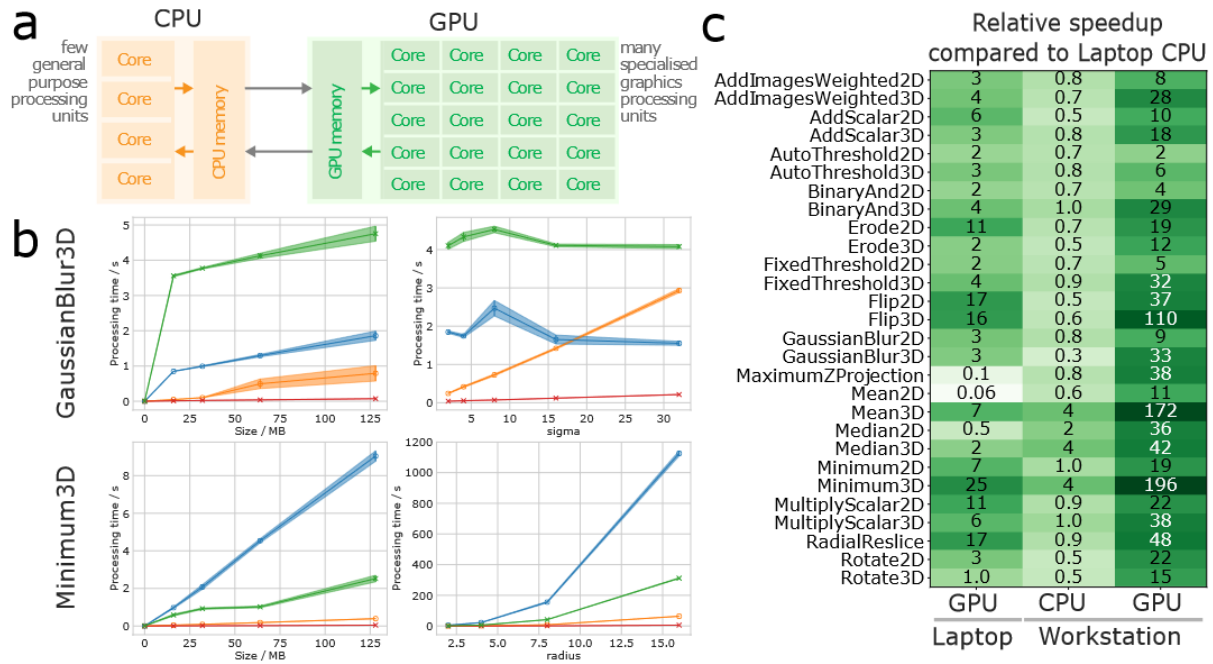
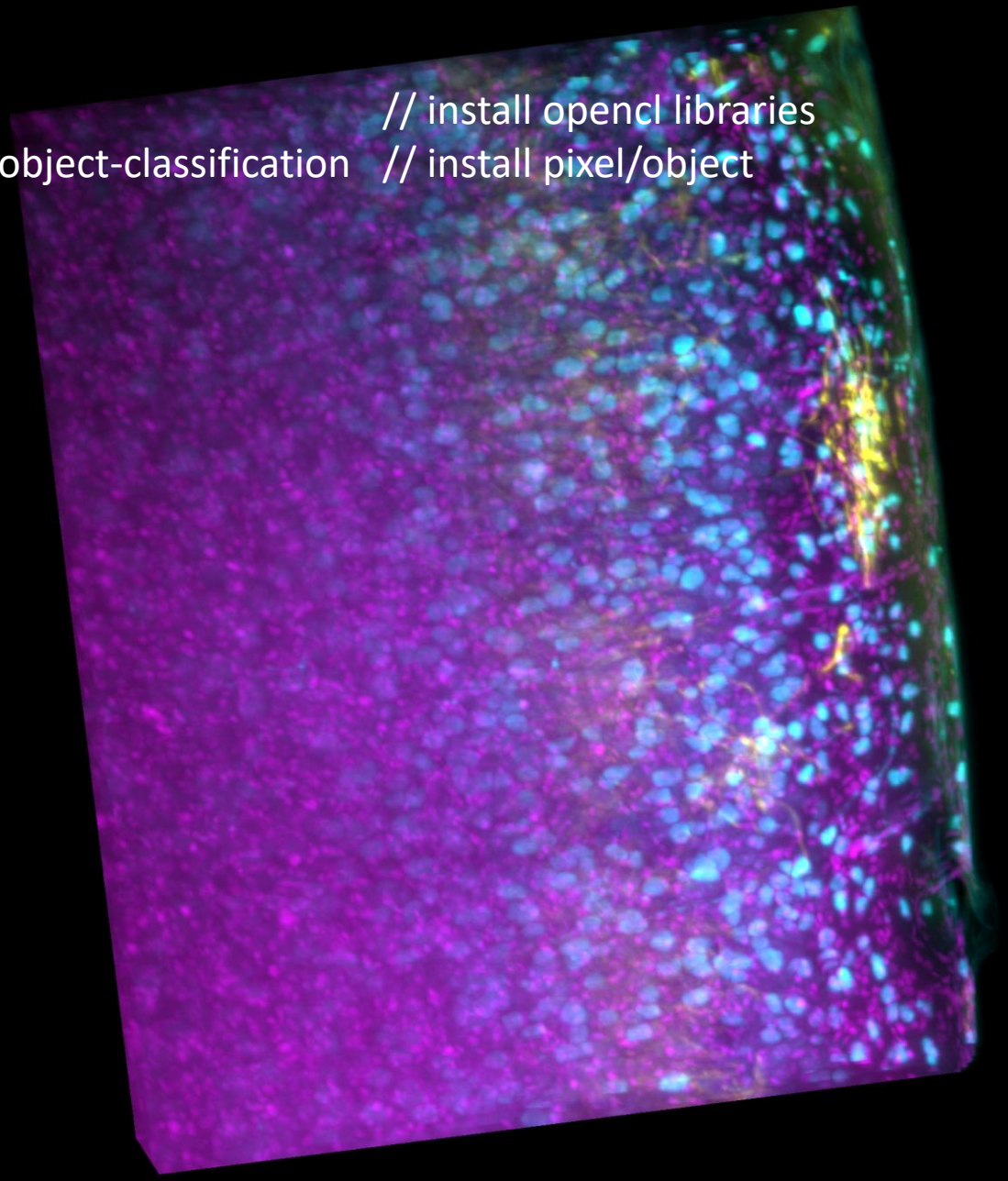


Figure 1: **(a)** GPU-acceleration schematically shows how many GPU cores potentially outperform a CPU with less cores. **(b)** Execution time of the Gaussian blur and minimum filter for different image sizes and parameters. Error bands denote the 99.9% confidence interval. **(c)** Overview of speed-up measurements when applied to 32 MB (2D) / 64 MB (3D) large 16-bit images with respect to computation times on a laptop CPU. Time measurements excluded memory transfer and compilation times.

In an **Miniforge prompt**:

- mamba install -c conda-forge pyopencl // install opencl libraries
- pip install pyclesperanto napari-accelerated-pixel-and-object-classification // install pixel/object classification and Napari assistant



In an **Miniforge prompt**:

- mamba install -c conda-forge pyopencl // install opencl libraries
- pip install pyclesperanto napari-accelerated-pixel-and-object-classification // install pixel/object classification and Napari assistant

In Napari:

- Open PR012_L.tif (available at https://www.ebi.ac.uk/biostudies/bioimages/studies/S-BIAD1077?query=brain%20microscopy%20fluorescence)

- Train a pixel classifier to segment myelinated neurites

- Train a pixel classifier to segment neuronal cell bodies